

审定成绩：_____

重庆邮电大学 毕业设计（论文）

中文题目	面向冷启动问题的内容推荐算法 模型设计与实现
英文题目	Design and Implementation of Content Recommendation Algorithm Model for Cold-start Problem
学院名称	计算机科学与技术学院/人工智能学院
学生姓名	敖宇
专 业	数据科学与大数据技术
班 级	04081901
学 号	2019212375
指导教师	黎俊伟 讲师
答 辩 组 负 责 人	龙林波 副教授

二〇二三年 六月
重庆邮电大学教务处制

计算机科学与技术/人工智能学院本科毕业设计(论文)

诚信承诺书

本人郑重承诺：

我向学院呈交的论文《面向冷启动问题的内容推荐算法模型设计与实现》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级 2019

专业 数据科学与大数据技术

班级 04081901

承诺人签名

年 月 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

学生签名：

指导老师签名：

日期： 年 月 日 日期： 年 月 日

摘要

过往的推荐算法无论是协同过滤、矩阵分解或是深度学习模型都是将用户的兴趣和物品的特征映射到一个隐空间中，并通过在该空间中的向量相似度计算来推荐物品，这种方法往往过度依赖于历史交互数据。而在现实生活中，我们常常会有源源不断的缺少历史行为交互的新用户、新物品加入，如何为新用户推荐物品，如何将新物品推荐给可能感兴趣的用户成为难题，而冷启动推荐即是为了解决很少或是没有历史交互数据的用户和物品的冷启动问题。

基于此，本文实现了一种不仅可以适用于热启动还可以适用于冷启动的算法，在冷启动问题表现不错的 DropoutNet 算法的基础上，不再使用其他模型预训练好的向量做为用户、物品隐向量的初始化，而是通过高斯随机初始化用户、物品的隐向量表示，降低了模型使用门槛，在训练过程中使用随机丢弃技术模拟冷启动环境，使其不仅可以在热启动环境中表现良好，遇到冷启动环境也能取得不错的推荐效果。

为检验此算法的性能好坏，本工作在 Flickr 和 Yelp 数据集上，利用 HR 和 NDCG 评测指标针对模型的性能进行测试。在热启动环境下，通过与 BPR、PMF 模型进行性能对比来衡量该模型好坏。冷启动环境下，通过改变模型内部超参数、神经网络层数等来衡量模型性能好坏。

关键词：推荐系统，冷启动问题，协同过滤，深度学习

Abstract

In the past, whether it is collaborative filtering, matrix factorization or deep learning model, the recommendation algorithm maps the user's interest and the characteristics of the item into a latent space, and recommends the item through the vector similarity calculation in this space. Methods tend to rely too heavily on historical interaction data. In real life, we often have a steady stream of new users and new items that lack historical behavior interaction. How to recommend items for new users and how to recommend new items to users who may be interested becomes a problem, and cold start recommendation that is to solve the cold start problem of users and items with little or no historical interaction data.

Based on this, this paper implements an algorithm that can be applied not only to hot start but also to cold start. Based on the DropoutNet algorithm that performs well in cold start problems, it no longer uses other models pre-trained vectors as user the initialization of the hidden vector of the item, but through Gaussian random initialization of the hidden vector representation of the user and the item, lowers the threshold for using the model. During the training process, the random discarding technique is used to simulate the cold start environment, so that it can not only perform well in the hot start environment, it can also achieve good recommendation results in cold start environments.

In order to test the performance of this algorithm, this work uses the HR and NDCG evaluation indicators to test the performance of the model on the Flickr dataset and Yelp dataset. In the hot start environment, the performance of the model is compared with the BPR model and PMF model to measure the quality of the model. In the cold start environment, the performance of the model is measured by changing the internal hyperparameters of the model, the number of neural network layers, etc.

Keywords: Recommender system, Cold-start problem, Collaborative filtering, Deep learning

目录

第 1 章 引言	1
1.1 研究背景和意义	1
1.2 国内外研究现状	2
1.2.1 推荐系统研究现状	2
1.2.2 冷启动推荐研究现状	4
1.3 主要内容和工作安排	6
第 2 章 相关知识基础	7
2.1 基于内容的推荐算法	7
2.2 协同过滤	8
2.2.1 基于内存的协同过滤	8
2.2.2 矩阵分解	9
2.2.3 概率矩阵分解	10
2.3 深度学习基础	11
2.3.1 神经网络	11
2.3.2 激活函数	11
2.3.3 梯度下降算法	13
2.3.4 深度学习框架	14
2.4 贝叶斯个性化排序	14
2.5 DropoutNet	15
2.5.1 Dropout 技术	15
2.5.2 DropoutNet	16
2.6 本章小结	17
第 3 章 面向冷启动问题的内容推荐算法	19
3.1 问题描述	19
3.2 算法设计	19
3.2.1 模型介绍	19
3.2.2 模型构建	20

3.2.3 损失函数设计	22
3.2.3 模型训练	22
3.3 本章小结	24
第 4 章 实验结果与分析	25
4.1 实验数据	25
4.1.1 数据来源	25
4.1.2 数据集预处理及划分	25
4.2 性能评测	27
4.2.1 评测方法	27
4.2.2 评测指标	27
4.3 模型超参数	28
4.4 模型性能	29
4.4.1 实验方案	29
4.4.2 热启动性能	30
4.4.3 冷启动性能	32
4.5 本章小结	36
第 5 章 总结与展望	37
5.1 主要工作	37
5.2 后续工作	37
参考文献	39
致谢	43
附录 A 部分源代码	45
附录 B 英文原文	49
附录 C 英文翻译	59

第 1 章 引言

1.1 研究背景和意义

随着互联网数据规模的不断增大，人们面临的信息量越来越大，而且这些信息可能是琳琅满目、五花八门的，用户在信息过载的情况下，很难有效地找到自己需要的信息，同时用户个性化需求的日益增强使传统的信息检索方法已经无法满足需求，因此，推荐系统作为一种个性化的信息提供方式应运而生^[1]。推荐系统是一种利用用户行为数据和偏好信息，通过算法进行分析和处理，从而向用户推荐可能感兴趣的产品或内容的技术，它可以帮助用户快速发现自己感兴趣的信息、商品、服务等，并提高用户的体验满意度，其广泛应用于电商、社交网络、新闻资讯、视频网站等领域，如图 1.1 所示。在电商领域，推荐系统可以根据用户浏览和购物历史、搜索记录等，向用户推荐可能感兴趣的商品和服务，提高用户转化率和购买满意度。在社交网络和新闻资讯领域，推荐系统可以根据用户的兴趣、朋友圈、阅读历史等，向用户推荐相关的内容，提高用户对平台的活跃度和留存率。



图 1.1 推荐系统应用场景

然而，在实际应用中，推荐系统面临着一个严峻的问题，即冷启动问题^[2]，如图 1.2 所示。该问题指的是在一个新产品或新用户进入推荐系统时，由于缺少相应的历史数据，无法进行有效的推荐，从而导致用户体验下降，甚至给推荐系统的商业效益带来威胁。

图 1.2 推荐系统冷启动问题¹

如果推荐系统无法应对冷启动问题，会导致以下几个方面的问题：

- a) 用户流失。对于新用户来说，如果推荐系统不能推荐出他们需要的产品或服务，他们可能会选择放弃使用该系统，转而寻找其他的解决方案。
- b) 销售额下降。对于新品牌来说，如果他们无法通过推荐系统向目标用户进行定向营销，可能会导致销售额下降，甚至影响企业的长期发展。
- c) 推荐系统性能下降。冷启动问题也会导致推荐系统的性能下降，这是因为对于缺乏信息的新用户或新物品，推荐系统无法建立有效的联系和关系，导致推荐结果不准确，从而影响用户的体验和满意度。

因此，解决冷启动问题是推荐系统发展中亟待解决的问题，需要从多方面进行改进和优化，提高推荐系统的性能和精度。本文通过学习 DropoutNet 算法^[3]的思想，再通过高斯随机初始化用户、物品隐向量，在神经网络模型训练时使用丢弃法模拟冷启动环境进行训练，使得模型在推理阶段不仅可以在热启动环境下取得良好的效果还可以适用于冷启动环境，使得新用户和新物品能够更好的被推荐和推荐。

1.2 国内外研究现状

1.2.1 推荐系统研究现状

推荐系统的发展可以分为传统的推荐算法和基于深度学习的推荐算法两个阶段^[4]。

1) 传统的推荐算法

在传统的推荐算法阶段，基于内容过滤的推荐算法（Content-based Recommendation，简称 CB）^[5]和基于协同过滤的推荐算法（Collaborative-filtering

¹ <https://zhuanlan.zhihu.com/p/376798647>

Recommendation, 简称 CF)^[6]广受欢迎。基于内容过滤的推荐算法是一种利用物品的属性信息进行推荐的方法, 它通过对用户历史浏览记录和已喜欢的物品进行提取特征, 计算相似度来为用户推荐与已有兴趣相关的物品。而基于协同过滤的推荐算法则是通过分析历史评分矩阵以及用户之间的相似度或物品之间的相似度来得到用户与项目之间的关系, 进而预测新用户与项目之间的关联关系, 来推荐未知的物品或者其他用户也感兴趣的物品。协同过滤算法又包括基于内存的协同过滤和基于模型的协同过滤。基于内存的协同过滤根据用户行为数据计算相似度并直接利用这些相似度进行推荐, 其中最为经典的算法包括用户-用户协同过滤和物品-物品协同过滤, 其缺点是需要庞大的内存来储存相似度矩阵。基于模型的协同过滤最经典的算法就是矩阵分解 (Matrix Factorization, 简称 MF)^[7], 包括概率矩阵分解 (Probabilistic Matrix Factorization, 简称 PMF)^[8]和奇异值分解 (Singular Value Decomposition, 简称 SVD)^[9], 及一些改进版本 Funk SVD、RSVD、SVD++ 等等, 相比于基于内存的协同过滤其优点为需要的存储较小, 只需要存储用户、物品隐向量矩阵。总体来说, 基于协同过滤的推荐算法具有计算简单、易于实现、效果良好等优点, 但是也存在冷启动问题、稀疏性问题、可扩展性差等缺点。

在后来的发展中, 混合推荐 (Hybrid Recommendation, 简称 HR)^[10]成为了一种流行的推荐方式, 它涵盖了多种推荐算法, 如基于内容过滤和协同过滤相结合的混合算法、基于标签的推荐算法、基于时间的推荐算法等等。混合推荐简单来说就是将不同的推荐算法进行组合, 从而提高推荐效果, 其优点是一定程度上克服了数据稀疏、弥补了不同技术缺点, 但其也存在缺少高效的混合模式、推荐过程较复杂等缺点。

2) 基于深度学习的推荐算法

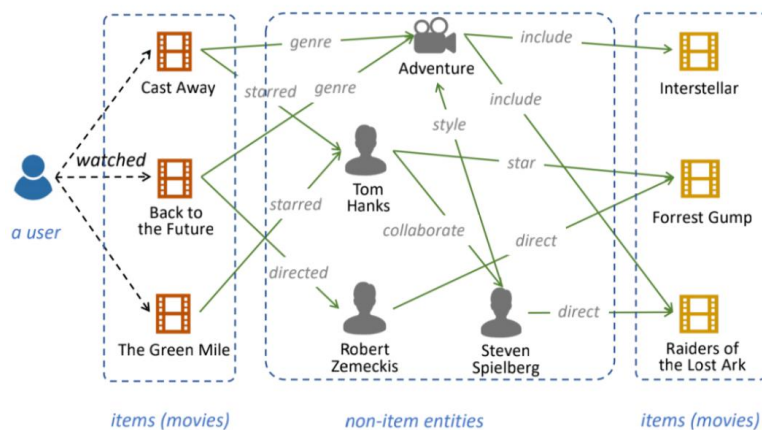
近年来, 随着深度学习的发展, 基于深度学习的推荐算法受到了广泛的关注。其核心思想是利用深度神经网络 (Deep Neural Network, 简称 DNN)^[11]、循环神经网络 (Recurrent Neural Network, 简称 RNN)^[12]、卷积神经网络 (Convolutional Neural Networks, 简称 CNN)^[13]、图神经网络 (Graph Neural Network, 简称 GNN)^[14]等各种深度学习模型来进行推荐。DNN 具有处理非线性关系的能力, 因此可以很好地解决传统推荐算法处理非线性数据的问题, 基于 DNN 的推荐算法主要通过学习用户和商品之间的显式和隐式特征, 来实现准确的推荐。RNN 是一种能够处

理序列数据的神经网络模型，可以很好地处理推荐系统中的时间序列数据，基于 RNN 的推荐算法可以通过学习用户行为序列中的时间信息来预测用户下一步可能的行为。CNN 可以用来学习排序模型，以便根据用户的偏好和历史行为进行个性化推荐，与此同时，对于多源输入的情况，CNN 可以同时考虑图片、文本和用户行为数据等多种信息来源，实现更全面、更准确的多模态推荐。推荐系统往往需要从多个方面考虑用户的偏好信息，如用户之间的关系、用户历史行为记录等。这些信息可以表示为一个用户行为图的形式，GNN 可以有效地处理这种图结构数据，来计算用户之间的相似度，并进行更加准确的推荐^[15]。

与传统推荐算法相比，基于深度学习的推荐算法具有更多优点，它不但可以处理非线性关系，捕捉更高级别的特征和模式，从而提高推荐精度，还可以利用大量的标记和未标记的数据进行训练，从而增加特征信息，在处理稀疏数据上更有效。基于深度学习的推荐算法还可以同时使用用户行为数据、文本、图片、音频、视频等不同类型的数据源进行训练，从而得到更准确的推荐结果。

1.2.2 冷启动推荐研究现状

目前，针对推荐系统中的冷启动问题，往往要使用到用户或者物品的属性特征以及额外的补充信息进行建模^[16]。DropoutNet 提出了一种神经网络模型，利用其他模型预训练好的用户、物品隐向量表示以及属性特征，在训练时通过对隐向量随机置零模拟冷启动环境，进而提高模型的鲁棒性。MetaEmbedding^[17]提出将用户和物品的特征嵌入到一个全局共享的元空间中，从而充分利用已有的数据信息，并为新用户或新物品提供一种可行的表示方式再做后续的训练及预测。相比于其他冷启动解决方法，MetaEmbedding 具有以下优点：一是可以充分利用已有数据的信息，减轻冷启动的影响；二是可以提高推荐的准确性和场景适应性；三是可以在实际应用中进行在线更新和动态调整，不断加强模型的泛化能力。同时，还可以使用现有数据构建知识图谱^[18]来解决冷启动问题，利用知识图谱^[19]的背景知识和数据，通过图谱嵌入技术构建一个元空间^[20]，将实体和属性嵌入到该空间中，再在该空间中计算实体之间的相似度和相关性，提供推荐系统所需的信息，如图 1.3 所示。

图 1.3 电影推荐系统中的知识图谱^[21]

文章^[22]介绍了一种基于对比学习的方法来解决冷启动推荐问题。该方法首先利用已有的交互数据和内容数据来预测用户的隐式反馈信息，然后通过对比学习将隐式反馈信息嵌入到低维空间中，最终利用嵌入向量计算得到用户和物品之间的相似度。文章通过寻找最近邻的方法来评估推荐结果，同时在不同的数据集上进行了实验，证明了该方法在冷启动推荐任务上的有效性。

文章^[23]介绍了如何利用链接开放数据（Linked Open Data, LOD）技术来解决基于协同过滤的推荐系统中的数据稀疏性和冷启动问题。文章提出了一种 LODCF 方法，该方法利用 LOD 中的实体关系构建用户和物品的语义表示，并将其与传统矩阵分解相结合，形成一个联合推荐模型。在此基础上，文章还提出了一种基于实体的加权矩阵分解算法，以进一步提高推荐效果。实验结果表明，与其他方法相比，LODCF 方法能够在大规模真实世界数据集上显著提高推荐准确度，并且能够很好地解决数据稀疏性和冷启动问题。

方阳, 谭真等人, 在文章^[24]中提出了一种名为 CM-HIN 的基于异质信息网络^[25]的推荐系统对比元学习框架, 其主要目的是克服推荐系统中的冷启动问题。因为传统的推荐模型难以充分利用异质信息网络提供的丰富和有意义的辅助和语义信息, 并容易忽略网络中的无标签信息, 为了解决这个问题, CM-HIN 框架使用元路径和网络模式这两个视图建模异质信息网络中的高阶和本地结构信息, 并采用对比学习方法挖掘网络中的无标签信息, 最终将这两个视图整合在一起, 通过大量实验表明 CM-HIN 能够显著提高冷启动场景下推荐系统的性能, 超越其他先进的基线模型。

总之，针对不同场景和数据冷启动情况，往往需要采用不同的算法和策略进行解决。未来的研究方向之一就是建立更为有效的冷启动推荐算法和模型，以提高推荐系统的实际应用效果。

1.3 主要内容和工作安排

本文在 DropoutNet 算法思想的基础上，将其训练时使用丢弃法模拟冷启动环境进行训练的方法用在 BPR 上，实现了一种通过高斯随机初始化方法来初始化用户、物品隐向量表示的推荐算法，让 BPR 不仅可以适用于热启动推荐系统还可以在冷启动推荐系统中取得不错的效果。

本文工作安排如下：

第 1 章首先介绍了本毕设的研究背景和意义，再对推荐系统、冷启动推荐的发展历程与研究现状从算法角度进行了简述，探讨了冷启动推荐未来的发展趋势和挑战。

第 2 章介绍本文将要用到的相关知识，包括多种推荐算法（矩阵分解、PMF、BPR、DropoutNet 等）、深度学习基础（梯度下降、激活函数等）、开源深度学习框架 TensorFlow 等。

第 3 章详细介绍了本文所实现的面向冷启动问题的内容推荐算法的具体内容，包括算法模型的设计与训练方式等。

第 4 章介绍实验所用到的数据集及预处理方式，同时确定了性能评测指标，对比了该模型与其他模型热启动情况下的性能，此外还分析了相同模型下不同超参数、神经网络层数对冷启动性能的影响。

第 5 章，对本毕设的所有工作进行了综述并提出了当前工作的特点和存在的不足。此外，还探讨了未来工作的研究方向，以帮助进一步改进冷启动推荐系统。

第2章 相关知识基础

2.1 基于内容的推荐算法

基于内容的推荐算法（Content-based Recommendation，简称 CB）是指根据物品的内容特征（如关键词、类别、标签等）来推荐相似内容给用户，如图 2.1 所示。

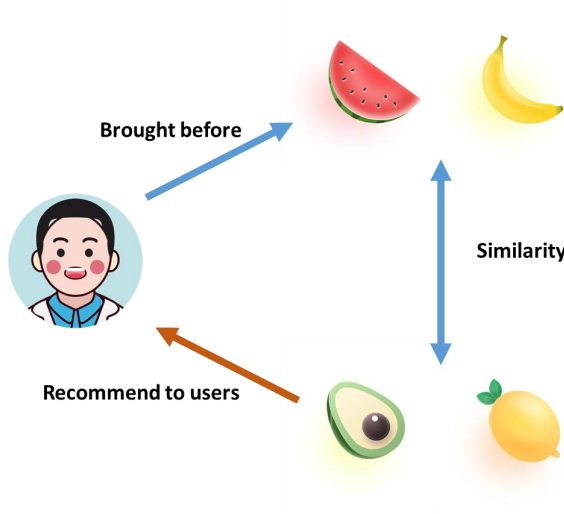


图 2.1 基于内容的推荐算法

其基本思想是，如果用户对某一种内容感兴趣，那么他们可能也会对相似的内容感兴趣。基于内容的推荐算法主要包括以下步骤：

- 特征提取：**将物品的内容特征提取出来并转化成向量表示，比如可以使用 TF-IDF^[26]、Word2Vec^[27]等方法。
- 相似度计算：**计算物品之间的相似度，常见的方法有余弦相似度、欧几里得距离等。公式 2.1 为余弦相似度计算，其中 $\mathbf{f}_i$ 表示第 i 个物品的特征向量，公式 2.2 为欧几里得距离计算，其中 $\vec{\mathbf{x}}$ 和 $\vec{\mathbf{y}}$ 表示两个物品的特征向量， x_i 和 y_i 分别表示 $\vec{\mathbf{x}}$ 和 $\vec{\mathbf{y}}$ 在第 i 个维度上的分量，距离越小表示两物品的相似度越高。

$$\cos(\mathbf{f}_i, \mathbf{f}_j) = \frac{\mathbf{f}_i \cdot \mathbf{f}_j}{\|\mathbf{f}_i\|_2 \times \|\mathbf{f}_j\|_2} \quad (2.1)$$

$$d(\vec{\mathbf{x}}, \vec{\mathbf{y}}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.2)$$

- 推荐生成：**根据用户的历史行为以及物品的相似度，生成推荐列表。

基于内容的推荐算法的优点是可以推荐用户喜欢的新物品，而不需要考虑其他用户的行为数据。但是，该算法容易出现“过度推荐”问题，即推荐过于相似的物品，导致用户失去兴趣。因此，需要采用一些方法来增加推荐物品的多样性。

2.2 协同过滤

2.2.1 基于内存的协同过滤

基于内存的协同过滤（Memory-Based CF）通过分析用户历史行为数据和物品属性数据，来预测用户对某些物品的兴趣度。

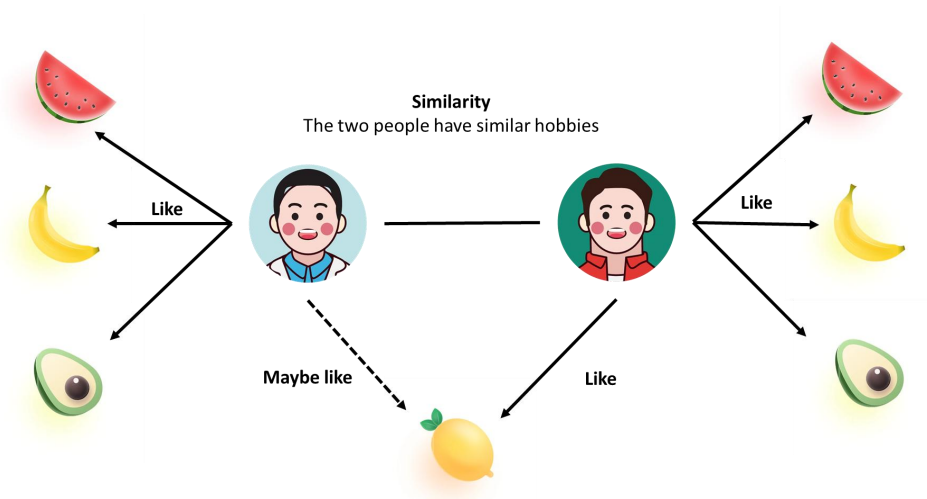


图 2.2 基于用户的协同过滤

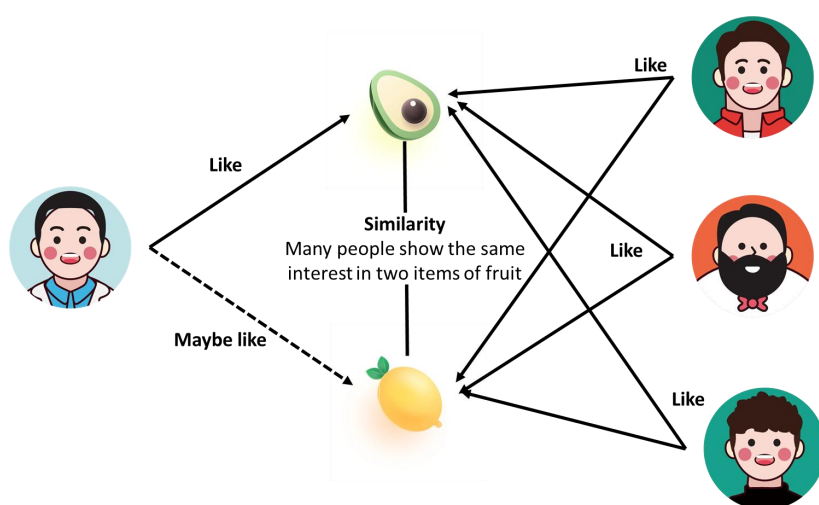


图 2.3 基于物品的协同过滤

依据用户和物品两个不同的维度基于内存的协同过滤又分为基于用户的协同过滤和基于物品的协同过滤，前者通过计算用户之间的相似性，然后根据与目标用户最相似的 k 个用户的历史行为数据，来预测目标用户可能喜欢的项目，如图 2.2 所示，后者则是首先计算项目之间的相似性，然后根据目标用户之前喜欢过的项目和这些项目相似的其他项目的历史行为数据，来预测目标用户可能喜欢的其他项目，如图 2.3 所示。

2.2.2 矩阵分解

矩阵分解（Matrix Factorization，简称 MF）也是一种基于协同过滤的算法，其核心思想是将原始的用户-物品评分共现矩阵分解为两个低维矩阵的乘积，从而识别出潜在的用户和物品的特征向量，用这些向量来预测用户对尚未评价的物品的评分，如图 2.4 所示。

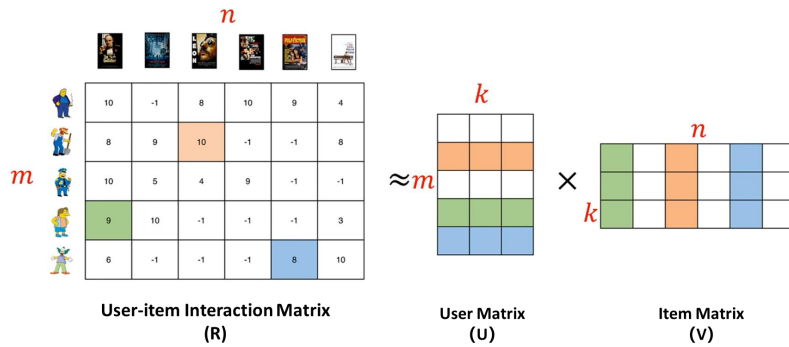


图 2.4 矩阵分解

具体来说，图中评分矩阵 $\mathbf{R} \in \mathbb{R}^{m \times n}$ 表示 m 个用户对 n 个物品的评分，其中 -1 表示用户没有对物品评分，通过将 $\mathbf{R}$ 分解为两个低维矩阵 $\mathbf{U} \in \mathbb{R}^{m \times k}$ 和 $\mathbf{V} \in \mathbb{R}^{n \times k}$ 的乘积，其中 k 表示潜在的特征数，也就是用来描述用户和物品的特征向量的维度。那么原有的评分矩阵可以得到如下的近似表示：

$$\hat{\mathbf{R}} = \mathbf{U}\mathbf{V}^T \quad (2.3)$$

$$\hat{r}_{u,i} = \mathbf{u}_u \mathbf{v}_i^T \quad (2.4)$$

其中 $\hat{\mathbf{R}}$ 表示评分矩阵的近似值，相应的， $\hat{r}_{u,i}$ 即为预测的用户 u 对物品 i 的评分。

矩阵分解通过最小化预测评分和实际评分之间的差距，如下公式 2.5 为最小化均方误差损失函数（Mean Squared Error）：

$$\min_{\mathbf{U}, \mathbf{V}} \sum_{(u,i) \in \mathbf{R}} (r_{u,i} - \hat{r}_{u,i})^2 \quad (2.5)$$

同时，为了防止矩阵分解的过拟合问题，可以在损失函数中加入正则化项对模型参数进行惩罚来减少复杂度，从而避免过拟合，带有正则化的损失函数为：

$$\min_{\mathbf{U}, \mathbf{V}} \sum_{(u,i) \in \mathbf{R}} (r_{u,i} - \hat{r}_{u,i})^2 + \lambda(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) \quad (2.6)$$

上述正则化项中 $\|\mathbf{U}\|_F^2$ 和 $\|\mathbf{V}\|_F^2$ 分别表示矩阵 $\mathbf{U}$ 和 $\mathbf{V}$ 的 Frobenius 范数，并可以理解成矩阵的平方和范数，这种正则化方式称为 L2 正则化^[28]。当正则化系数 λ 较大时，模型会更倾向于使用较小的权重值，从而减少模型的复杂度降低过拟合的风险。

最后，通过使用梯度下降等优化算法来最小化带有正则化项的损失函数，并更新矩阵 $\mathbf{U}$ 和 $\mathbf{V}$ 的值。

2.2.3 概率矩阵分解

概率矩阵分解（Probabilistic Matrix Factorization，简称 PMF）的目标同样是求出 $\mathbf{U}$ 和 $\mathbf{V}$ 矩阵，但 PMF 基于 $\mathbf{R}$ 和 $\hat{\mathbf{R}}$ 的差、 $\mathbf{U}$ 和 $\mathbf{V}$ 矩阵均服从高斯分布的假设，通过最大化对数似然函数的加权和来定义目标函数，并加上简单的正则化项：

$$\max_{\mathbf{U}, \mathbf{V}} \sum_{(u,i) \in \mathbf{R}} \omega_{u,i} \log p(r_{u,i} | \mathbf{u}_u, \mathbf{v}_i, \sigma^2) - \frac{\lambda}{2}(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) \quad (2.7)$$

其中 $\omega_{u,i}$ 表示预先定义的一个值，用于区分哪些是观测值，哪些是缺失值。 σ^2 是一个预定义的方差，代表噪声的程度，正则化参数 λ 用于控制模型的复杂度。

利用贝叶斯定理将先验概率加入之后，上述公式可以转化为最大后验估计（Maximum A Posteriori, MAP）^[29]的形式：

$$\max_{\mathbf{U}, \mathbf{V}} \sum_{(u,i) \in \mathbf{R}} \log p(r_{u,i} | \mathbf{u}_u, \mathbf{v}_i, \sigma^2) + \log p(\mathbf{U}) + \log p(\mathbf{V}) - \frac{\lambda}{2}(\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) \quad (2.8)$$

其中， $\log p(\mathbf{U})$ 和 $\log p(\mathbf{V})$ 是先验概率分布，可以使得模型更加稳定和鲁棒。在优化过程中，初始值是随机生成的，通过梯度下降等优化算法，进行多次运算以期达到全局最优解。

2.3 深度学习基础

2.3.1 神经网络

前馈神经网络（Feedforward Neural Network，简称 FNN）是一种最基本的神经网络模型，它由输入输出层及多个隐藏层组成。其中，每一层都由多个节点组成，并与下一层之间进行全连接。信息从输入层开始，通过每一层的非线性变换传递到下一层，最终到达输出层，其结构如下图 2.5 所示。

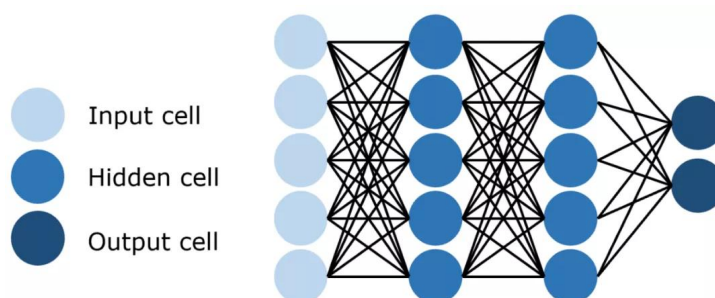


图 2.5 FNN 结构示意图

2.3.2 激活函数

为了防止神经网络退化为线性模型（层数塌陷），需要对每个隐藏单元应用非线性的激活函数（activation function） σ ，激活函数的输出 $\sigma(\cdot)$ 被称为活性值（activations），下面简单介绍几个常见的激活函数。

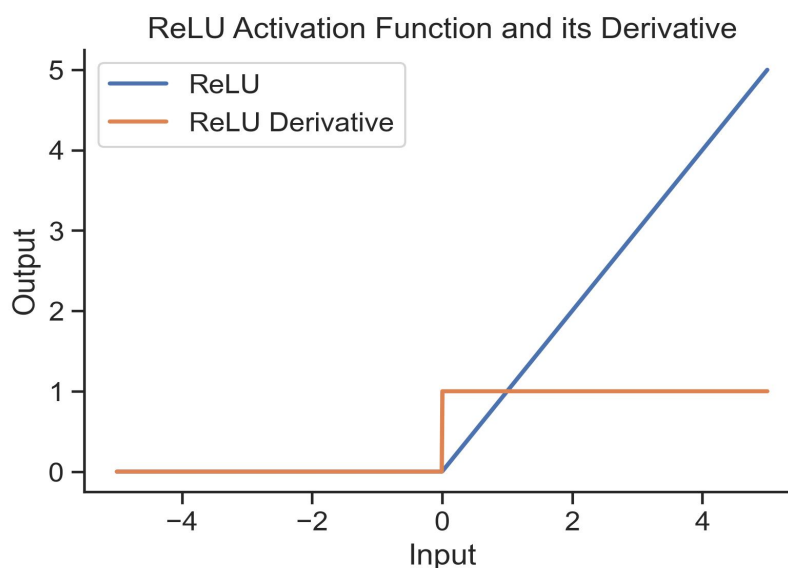


图 2.6 ReLU 函数及其导数

修正线性单元（Rectified linear unit, ReLU）^[30]是最受欢迎的激活函数，在于它实现简单且在多种任务上表现良好，ReLU 把给定的 x 与 0 比较并取最大值：

$$ReLU(x) = \max(x, 0) \quad (2.9)$$

其图像及导数图像如图 2.6 所示。

Sigmoid 函数也是一种常见的激活函数，它将范围 $(-\infty, +\infty)$ 中的任意输入压缩为区间 $(0, 1)$ 中的某个值，它和它的导数的公式如下：

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (2.10)$$

$$\frac{d}{dx} \text{sigmoid}(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = \text{sigmoid}(x)(1 - \text{sigmoid}(x)) \quad (2.11)$$

图像如图 2.7 所示：

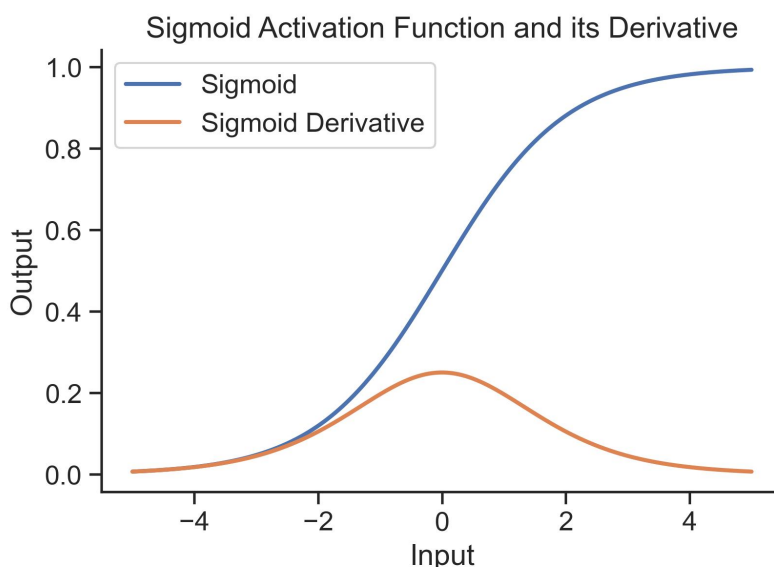


图 2.7 Sigmoid 函数及其导数

Tanh 被称为双曲正切函数，与 Sigmoid 函数类似，它能够将输入压缩到区间 $(-1, 1)$ 上，Tanh 函数及其导数的公式如下：

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}} \quad (2.12)$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (2.13)$$

图像如图 2.8 所示：

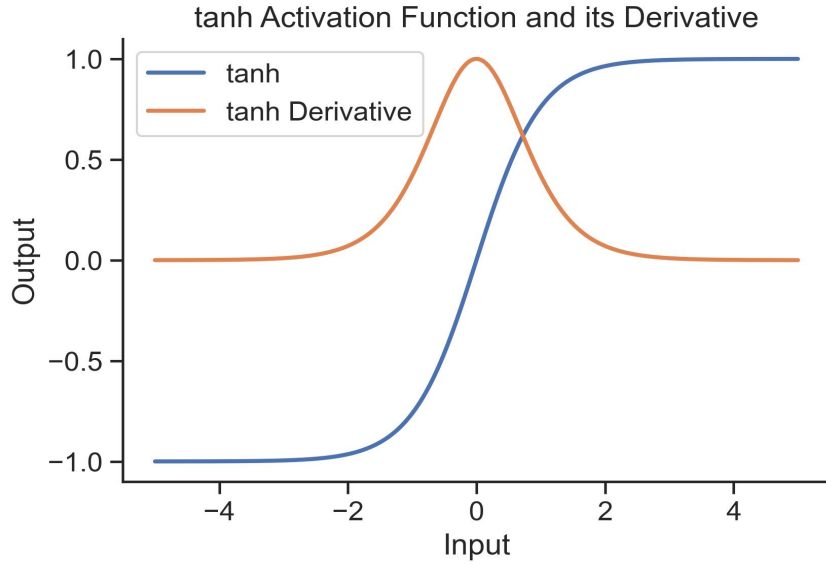


图 2.8 Tanh 函数及其导数

对比三者可以发现，Sigmoid 和 Tanh 的导数在输入值超过一定范围都会变得非常小，当神经网络层数过多时这会产生梯度消失问题从而导致模型收敛非常慢。而 ReLU 在正方向的导数恒为 1，这有利于缓解梯度消失问题，但在负方向上 ReLU 是采取的直接置为 0，这也可能导致有时权重无法更新的问题。

2.3.3 梯度下降算法

在深度学习过程中，通常需要利用优化算法最小化目标函数，而梯度下降算法^[31]则一种常用的优化算法，它可以在搜索过程中找到函数的局部最小值或全局最小值。其核心思想是利用目标函数的梯度信息进行迭代搜索，不断更新参数使得目标函数的值不断降低，直到达到最优解。

下面将介绍一种 Adam 算法 (Adaptive Moment Estimation, 自适应矩估计)^[32]，其被广泛运用于深度学习中，是一种高效、自适应性强且具有正则化作用的优化算法。它使用指数加权移动平均值^[33]来估算梯度的动量和二次矩，得到状态变量：

$$\begin{aligned} \mathbf{v}_t &\leftarrow \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1) \mathbf{g}_t \\ \mathbf{s}_t &\leftarrow \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \end{aligned} \quad (2.14)$$

同时为了防止初始偏差过大，对状态变量进行标准化：

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_1^t}, \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t} \quad (2.15)$$

重新缩放梯度得：

$$\mathbf{g}_t' = \frac{\eta \hat{\mathbf{v}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \varepsilon} \quad (2.16)$$

其中 η 为学习率，是由人为设定的步长用来解决收敛问题。为了防止分母为 0，通常取 $\varepsilon = 10^{-6}$ ，且能够保证数值稳定性和逼真度。最后写出更新方程为：

$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \mathbf{g}_t' \quad (2.17)$$

2.3.4 深度学习框架

随着深度学习的发展，许多开源深度学习框架相继涌现而出，包括 TensorFlow、PyTorch、Keras、Caffe 等等。不同的深度学习框架有者不同的优点和适用场景，选择合适的框架可以大大提高深度学习的研究和应用效率。本文选择的框架为 TensorFlow^[34]，其是由 Google Brain 团队开发的一款基于数据流图进行编程的深度学习框架，是目前应用最为广泛的深度学习框架之一。TensorFlow 支持分布式计算，可以在 CPU 和 GPU 上运行，它通过将计算表达式定义为数据流图，在运行时按照依赖关系自动进行计算并优化计算过程，从而实现对高效的数值计算和机器学习模型的构建和训练。

2.4 贝叶斯个性化排序

矩阵分解的训练集通常是 $\langle u, i, r \rangle$ 的三元组形式， r 表示用户 u 对物品的 i 的评分或者其他交互行为。这种 pointwise 策略把 r 为缺失值时当做负样本处理，但实际上这可能并不是负样本，只是暂时缺少交互而已。

贝叶斯个性化排序（Bayesian Personalized Ranking，简称 BPR）^[35] 的训练集则为 $\langle u, i, j \rangle$ 的三元组形式（也可表示为 $i >_u j$ ），这种基于 pairwise 思想的成对的序列表示对于用户 u 而言，在物品 i 和物品 j 中更偏好与 i 。BPR 本质上也是一种基于矩阵分解的排序算法，但它学习的是用户对不同物品的相对偏好关系，从而对物品进行排序再做推荐。

BPR 基于用户之间偏好、用户对不同物品之间偏好都相对独立的假设进行建模，目标是预测出对于用户集 U 和物品集 I 的排序矩阵 $\bar{\mathbf{R}} \in \mathbb{R}^{|U| \times |I|}$ ：

$$\bar{\mathbf{R}} = \mathbf{UV}^T \quad (2.18)$$

其中 $\mathbf{U} \in \mathbb{R}^{U \times k}$ 表示用户矩阵， $\mathbf{V} \in \mathbb{R}^{I \times k}$ 表示物品矩阵， k 表示用户、物品隐向量的长度。对于任意一个用户 u 和物品 i 都有：

$$\bar{r}_{ui} = \mathbf{u}_u \cdot \mathbf{v}_i^T = \sum_{f=1}^k u_{uf} \cdot v_{if} \quad (2.19)$$

BPR 基于最大后验估计 $P(\theta | >_u)$ ， θ 表示模型参数即 $\mathbf{U}$ 和 $\mathbf{V}$ ，由贝叶斯公式和独立性假设有：

$$P(\theta | >_u) \propto P(>_u | \theta) P(\theta) \quad (2.20)$$

公式 2.20 右边第一部分可写为：

$$\prod_{u \in U} P(>_u | \theta) = \prod_{(u, i, j) \in D} \sigma(\bar{r}_{ui} - \bar{r}_{uj}) \quad (2.21)$$

其中 $\sigma(x)$ 为 Sigmoid 函数，用于将输入映射为概率值，公式 2.20 右边第二部分则服从正态分布。最终得出最大对数后验估计函数：

$$\prod_{(u, i, j) \in D} \ln \sigma(\bar{r}_{ui} - \bar{r}_{uj}) + \lambda \|\theta\|^2 \quad (2.22)$$

对 $\mathbf{U}$ 和 $\mathbf{V}$ 进行随机初始化后，使用优化算法即可求出收敛后的模型参数，进而得出排序矩阵 $\bar{\mathbf{R}}$ 。BPR 算法不仅仅只关注用户对物品的评分，还可以利用用户的兴趣偏好信息进行建模，这些偏好信息可以更加准确地描述用户的兴趣爱好，从而提高推荐准确率。总之，BPR 算法相比于传统的矩阵分解算法，在准确率、处理隐式反馈数据、用户兴趣建模以及计算效率等方面具有明显优势，并在推荐系统中得到了广泛的应用。

2.5 DropoutNet

2.5.1 Dropout 技术¹

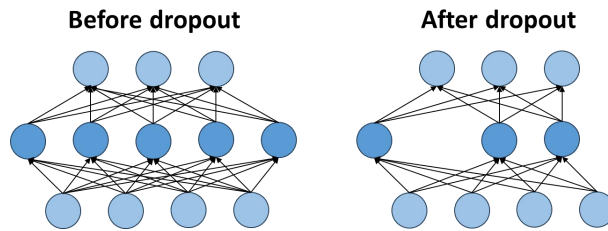


图 2.9 Dropout 技术

¹ https://zh-v2.d2l.ai/chapter_multilayer-perceptrons/dropout.html

Dropout 技术^[36]是一种用于防止模型过拟合的正则化技术，它在训练神经网络的前向传播中运用，通过一定概率随机丢弃一些神经元，从而使模型在训练时不会过于依赖某些特定的神经元而过拟合，如图 2.9 所示。

在标准的 Dropout 技术中，每一个中间活性值 $\mathbf{x}$ 以概率 p 置为 $\mathbf{0}$ ，且保持前后期望不变，即 $E[\mathbf{x}'] = \mathbf{x}$ ：

$$\mathbf{x}' = \begin{cases} \mathbf{0} & \text{概率为 } p \\ \frac{\mathbf{x}}{1-p} & \text{其他} \end{cases} \quad (2.23)$$

值得注意的是，Dropout 技术只在训练时开启而推理时应该关闭。因为在做推理时是为了获得最佳性能，应该使用所有神经元，而不是为了减少过拟合而牺牲一部分性能。

2.5.2 DropoutNet

DropoutNet 是一种能够处理推荐系统中冷启动问题的神经网络模型，冷启动问题无非就是新用户或者新物品进入系统时没有好的初始偏好隐向量表示。DropoutNet 则通过利用 Dropout 技术的思想，在训练时将输入随机替换以模拟冷启动环境，以便冷启动用户或物品进入模型能够有良好的推荐结果，模型示意图如下图 2.10 所示：

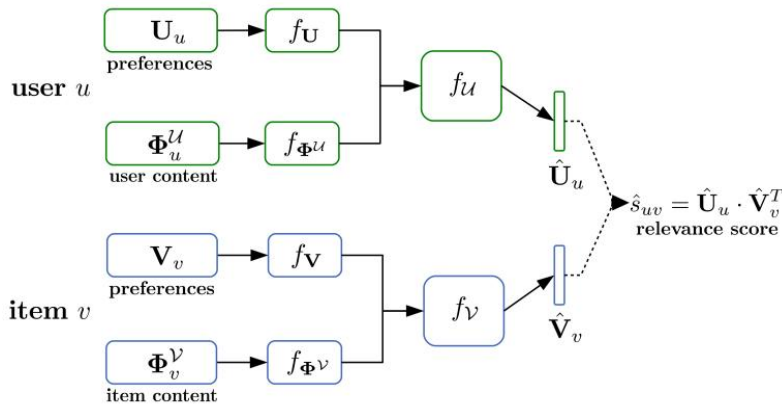


图 2.10 DropoutNet 结构图

DropoutNet 是一个双塔结构，以其他模型已经预训练好的偏好向量矩阵 U_u 和 V_v 以及 content 信息矩阵 Φ_u^U 和 Φ_v^V 做为模型输入，在经过全连接层 f ，得到 $\hat{U}_u$ 和 $\hat{V}_v$ 以获得最终得分，目标函数则写为：

$$\sum_{u,v} (\mathbf{U}_u \mathbf{V}_v^T - \hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T)^2 \quad (2.24)$$

在训练时，为了模拟冷启动环境，模型输入除了图 2.9 的四种向量表示全部输入以外，还会以一定几率改变输入形式为如下之一：

$$\text{A: } [\mathbf{U}_u, \Phi_u^u] \rightarrow [\mathbf{0}, \Phi_u^u]$$

$$\text{B: } [\mathbf{V}_v, \Phi_v^v] \rightarrow [\mathbf{0}, \Phi_v^v]$$

$$\text{C: } [\mathbf{U}_u, \Phi_u^u] \rightarrow [\text{mean}_{v \in \mathcal{V}(u)} \mathbf{V}_v, \Phi_u^u]$$

$$\text{D: } [\mathbf{V}_v, \Phi_v^v] \rightarrow [\text{mean}_{u \in \mathcal{V}(v)} \mathbf{U}_u, \Phi_v^v]$$

其中 A 和 B 将用户或物品的偏好向量随机置 $\mathbf{0}$ ，C 和 D 则是取对应均值做为新用户或新物品偏好向量的初始值，通过在每个 mini-batch 使用不同的输入策略进行训练，直到模型收敛为止。

2.6 本章小结

本章对实验所涉及的基础知识进行了基本介绍，包括推荐系统基本知识：基于内容的推荐、协同过滤、矩阵分解、PMF 等，并对深度学习的开源框架、激活函数、梯度下降算法等基本知识做了介绍，随后则介绍了 BPR、DropoutNet 等推荐算法。

第3章 面向冷启动问题的内容推荐算法

3.1 问题描述

端到端学习^[37]是指将整个模型作为一个整体进行训练，同时从原始数据开始，自动地学习特征表示，使得整个模型可以直接接受原始数据作为输入，输出所需的结果，中间不需要穿插其他过程。

DropoutNet 在训练时使用丢弃技术，已经在冷启动推荐方面取得不错效果，但其部分输入 $\mathbf{U}_u$ 和 $\mathbf{V}_v$ 为其他模型已经预训练好的用户、物品的偏好向量矩阵。预训练模型的性能好坏直接影响了后续的训练，不能实现端到端的学习，一定程度上提升了模型的使用门槛。

3.2 算法设计

3.2.1 模型介绍

为了降低冷启动推荐算法模型的使用门槛，本文旨在使用 DropoutNet 的训练方法，在 BPR 算法的基础上进行训练，不再使用其他预训练模型已经训练好的偏好向量表示，而是通过高斯随机初始化偏好向量进行训练，从而实现端到端的学习。与传统 BPR 算法不同的是，模型输入端不仅包括随机初始化的用户、物品偏好隐向量还包括用户、物品的内容信息，在保持 BPR 算法在热启动推荐中的优秀表现的同时，让 BPR 算法也能应用于冷启动推荐之中。其模型结构示意图如下图 3.1 所示：

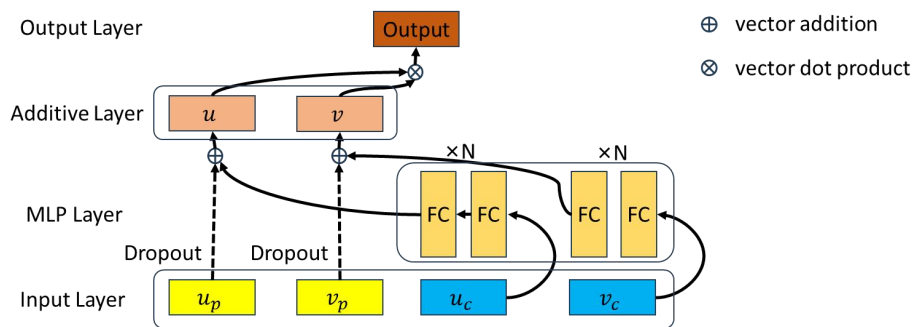


图 3.1 算法网络结构

网络各向量含义如下表 3.1 所示：

表 3.1 网络向量含义

向量名称	向量含义
$\mathbf{u}_p$	用户的偏好隐向量
$\mathbf{v}_p$	物品的偏好隐向量
$\mathbf{u}_c$	用户的内容信息向量
$\mathbf{v}_c$	物品的内容信息向量
$\mathbf{u}$	用户偏好及内容综合表示
$\mathbf{v}$	物品偏好及内容综合表示

训练时，网络模型中的 $\mathbf{u}_c$ 和 $\mathbf{v}_c$ 向量始终保持原始输入，而 $\mathbf{u}_p$ 和 $\mathbf{v}_p$ 则在随机初始化后利用 DropoutNet 算法中的丢弃操作，以一定概率置为 $\mathbf{0}$ 以模拟冷启动环境，以便模型能适用于冷启动用户或物品做为输入。

3.2.2 模型构建

模型总体设计包括四个部分：Input Layer、MLP Layer、Add Layer、Output Layer，其中除了第一层外的每一层输入均来自上一层的输出结果。

1) 输入层（Input Layer）

输入层的第一部分为用户偏好向量 $\mathbf{u}_p$ 和物品偏好向量 $\mathbf{v}_p$ ，其并不是真正意义上的输入进入模型的，输入模型的只是相对应的用户、物品索引，模型内部自动从 m 个用户和 n 个物品的高斯随机生成矩阵 $\mathbf{U}_p \in \mathbb{R}^{m \times k}$ 和 $\mathbf{V}_p \in \mathbb{R}^{n \times k}$ 中根据这些索引抽取出向量，这些向量的值在模型训练时是可以改变的。同时，为了方便完成训练时 $\mathbf{u}_p$ 、 $\mathbf{v}_p$ 的按概率置 $\mathbf{0}$ 操作，这里将两随机生成的偏好矩阵扩充一行，并把此行的值设置为全 $\mathbf{0}$ ，长度为 k ，同时停止此行的梯度更新以一直保持为全 $\mathbf{0}$ ，扩充后的矩阵形状为 $\mathbf{U}_p \in \mathbb{R}^{(m+1) \times k}$ 和 $\mathbf{V}_p \in \mathbb{R}^{(n+1) \times k}$ ，具体结构如式 3.1：

$$\mathbf{U}_p = \begin{pmatrix} a_{11} & \cdots & \cdots & \cdots & a_{1k} \\ \vdots & & & & \vdots \\ \vdots & & & & \vdots \\ a_{m1} & \cdots & \cdots & \cdots & a_{mk} \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix}, \mathbf{V}_p = \begin{pmatrix} a_{11} & \cdots & \cdots & \cdots & a_{1k} \\ \vdots & & & & \vdots \\ \vdots & & & & \vdots \\ a_{n1} & \cdots & \cdots & \cdots & a_{nk} \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix} \quad (3.1)$$

输入层的第二部分为用户的内容信息（包括性别、年龄、生日等属性）和物品的内容信息（包括颜色、类别、大小等属性），这部分为模型真正需要从模型外部进行输入。在训练过程中，这些信息的值做为输入不会被改变，只会改变其上连接的神经网络中的权重。内容信息向量的长度不一定为 k ，这里把长度记为 k' ，具体结构如式 3.2:

$$\mathbf{U}_c = \begin{pmatrix} a_{11} & \cdots & a_{1k'} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mk'} \end{pmatrix}, \mathbf{V}_c = \begin{pmatrix} a_{11} & \cdots & a_{1k'} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nk'} \end{pmatrix} \quad (3.2)$$

由上式中的矩阵 $\mathbf{U}_c \in \mathbb{R}^{m \times k'}$ 和 $\mathbf{V}_c \in \mathbb{R}^{n \times k'}$ ，在每次迭代时，根据行索引即可选择对应的物品和用户内容信息向量输入模型。

2) 神经网络层 (MLP Layer)

这一层的作用是通过全连接层把长度为 k' 的内容信息向量 $\mathbf{u}_c$ 和 $\mathbf{v}_c$ 变为长度为 k 的向量 $\mathbf{u}'_c$ 和 $\mathbf{v}'_c$ ，以便能和偏好向量做按位加法，同时因为内容信息为输入，不是变量，加入全连接层还可以自动提取内容信息。为了提高网络的性能，还在每一层后加入 ReLU 或 Sigmoid 等激活函数来增加非线性性，具体结构如下图 3.2 所示。

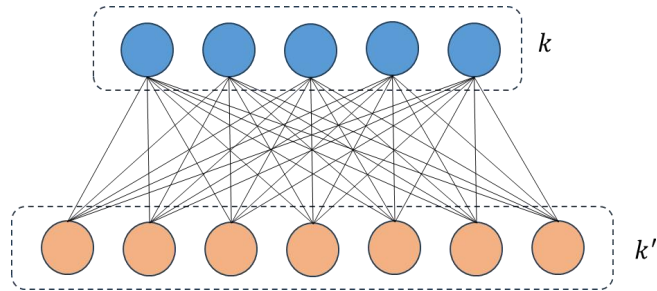


图 3.2 全连接层

计算公式如下:

$$y_i = \sigma\left(\sum_{j=1}^m w_{i,j} x_j + b_i\right) \quad (3.3)$$

其中 x_j 表示输入向量 $\mathbf{x}$ 中的第 j 个分量， y_i 表示输出向量 $\mathbf{y}$ 的第 i 个分量， $\sigma(x)$ 表示激活函数。

3) 加法层 (Additive Layer)

通过上一层的运算，长度为 k' 的内容信息向量 $\mathbf{u}_c$ 和 $\mathbf{v}_c$ 已变为长度为 k 的向量

$\mathbf{u}'_c$ 和 $\mathbf{v}'_c$ ，本层的主要任务是利用 `tf.add()` 将用户和物品的偏好向量、长度为 k 内容信息向量分别按位相加，形成最终的用户隐向量 $\mathbf{u}$ 和物品隐向量 $\mathbf{v}$ 的表示，其计算公式如下：

$$\mathbf{u} = \mathbf{u}'_c + \mathbf{u}_p \quad (3.4)$$

$$\mathbf{v} = \mathbf{v}'_c + \mathbf{v}_p \quad (3.5)$$

4) 输出层（Output Layer）

输出层为用户隐向量 $\mathbf{u}$ 和物品隐向量 $\mathbf{v}$ 做内积，并通过 Sigmoid 激活函数将结果映射至 0 到 1 之间，其计算公式为：

$$output = \sigma(\mathbf{u} \cdot \mathbf{v}^T) \quad (3.6)$$

3.2.3 损失函数设计

BPR 的目标函数是由三元组的数据集 $\langle u, i, j \rangle$ ，通过最大化用户 u 对于偏序对的偏好差异得来的，其最大对数后验估计形式可写为：

$$\prod_{(u,i,j) \in D} \ln \sigma(\bar{x}_{ui} - \bar{x}_{uj}) + \lambda \|\theta\|^2 \quad (3.7)$$

本文为了方便实验和使用梯度下降算法，使用的数据集的形式为 $\langle u, i, r \rangle$ 的三元组形式，其中 r 的取值仅为 $\{0, 1\}$ 。例如， $\langle u, i, 0 \rangle$ 和 $\langle u, j, 1 \rangle$ 则可等价于偏序对 $\langle u, j, i \rangle$ ，代表对于用户 u 而言，在 i 和 j 之中更偏向于 j 。故需要最小化的损失函数则可写为下式：

$$loss = \frac{1}{2} \sum_{i=1}^N (r - \hat{r})^2 \quad (3.8)$$

其中 N 表示数据集中正负样本的总数， r 表示真实标签， $\hat{r}$ 则表示模型输出即式 3.6 的结果。

3.2.3 模型训练

模型训练时，一个正样本随机采样多个负样本，以防止模型只学习到正样本的偏好信息。同时，训练时的负样本也需要进行更换，让模型不止学到个别负样本的信息，而是学到更多的负样本信息以增加模型的泛化性能。

在训练策略上,采用 DropoutNet 算法中的随机置0的方法来模拟冷启动环境,如下图 3.3 所示:

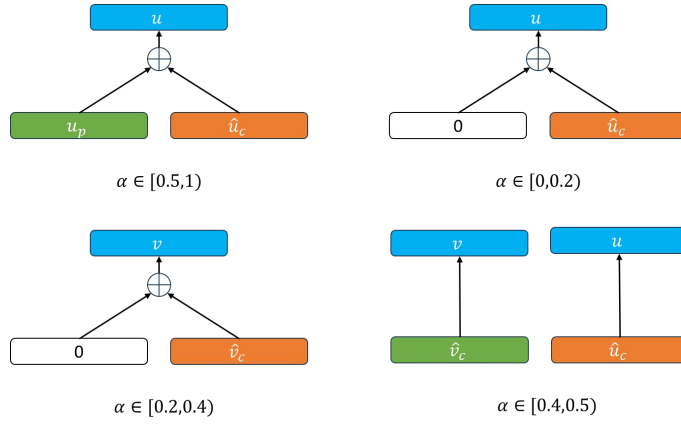


图 3.3 模型训练策略示意图

具体而言,模型在每一个 mini-batch 中,通过随机数模块 random 随机生成一个值 $\alpha \in [0,1)$,不同的 α 采用不同的训练输入:

- 1) 当 $\alpha \in [0,0.2)$ 时, u_p 进行置0操作,即 $u = 0 + \hat{u}_c, v = v_p + \hat{v}_c$;
- 2) 当 $\alpha \in [0.2,0.4)$ 时, v_p 进行置0操作,即 $u = u_p + \hat{u}_c, v = 0 + \hat{v}_c$;
- 3) 当 $\alpha \in [0.4,0.5)$ 时, u_p 和 v_p 均进行置0操作,即 $u = 0 + \hat{u}_c, v = 0 + \hat{v}_c$;
- 4) 当 $\alpha \in [0.5,1)$ 时,不进行置0操作,即 $u = u_p + \hat{u}_c, v = v_p + \hat{v}_c$;

算法 3.1 冷启动推荐近似算法步骤

Algorithm 3.1: Cold Start for BPR

Input: U_c, V_c
Initialize: user preference U_p , item preference V_p
While Not converged **do**:
 sample mini-batch $B = \{(i, j, 1), \dots, (m, n, 0)\}$
 apply one of:
 1. **leave as is:**
 $u = u_p + \hat{u}_c, v = v_p + \hat{v}_c$
 2. **user dropout:**
 $u = 0 + \hat{u}_c, v = v_p + \hat{v}_c$
 3. **item dropout:**
 $u = u_p + \hat{u}_c, v = 0 + \hat{v}_c$
 4. **user and item dropout:**
 $u = 0 + \hat{u}_c, v = 0 + \hat{v}_c$
 update U_p, V_p
until convergence
Output: u, v

虽然本算法是针对于冷启动推荐而设计，但为了保证热启动的良好性能，在算法中仍然设置了不置 $\mathbf{0}$ 操作，且不置 $\mathbf{0}$ 时的 α 跨度设置最大。

基于此，每一轮的每一个小批次都会根据随机生成数选择不同的模型内部操作进行训练。经过多轮迭代，模型即可适用于冷启动输入，其具体模型训练步骤如算法 3.1 所示。

3.3 本章小结

本章主要从问题描述、算法设计、模型训练三个方面对本文的主要工作进行了介绍，即在 BPR 的基础上采用 DropoutNet 的随机丢弃策略。在每个 mini-batch 中以一个随机生成数的大小选择不同的训练方式，让 BPR 在保持原有热启动性能的同时还能适用于冷启动。

第 4 章 实验结果与分析

4.1 实验数据

4.1.1 数据来源

Flickr¹是一个图片在线托管与社交网络服务网站，成立于 2004 年。用户可以在 Flickr 上上传、分享、组织和存储他们的照片，也可以浏览其他用户的作品，并与其他用户进行交流和互动。Flickr 的核心功能包括照片的上传、管理、搜索、标注和共享，同时也提供了多种社交网络和社区功能，例如评论、点赞、关注、定位和分类等，Flickr 对于摄影爱好者、设计师、艺术家和记者等人群来说都是一个非常有用的工具和社区。

Yelp²是一个在线的商家搜索和评论网站，提供了广泛的用户和商家信息。Yelp 网站允许用户对商家进行评分、写评论，并且可以搜索与自己有关的城市或商家。Yelp 的目标是通过用户提交的真实评论和其他信息（如地址、电话等）来帮助用户找到最好的商家，同时也帮助商家在当地市场获取更多曝光和生意。

本实验使用的第一个数据集是由德国的 Max-Planck-Institute for Software Systems 提供的一个基于 Flickr 上的用户-照片互动数据提取的评分数据集，其中包含了 8358 个用户对 82120 张照片的近 2500000 次评分，这个数据集可以用于推荐系统、机器学习、数据挖掘等领域的研究和应用。第二个数据集则是来自一个包含商家信息、用户信息和评论信息的大型社交媒体数据集，该数据集包含了来自 Yelp 网站的超过 500 万条评价、100 万个商家、200 万个用户和多个城市的地理位置信息。

4.1.2 数据集预处理及划分

1) 数据预处理

¹ <http://flickr.com/>

² <http://yelp.com/>

本实验下载到的数据集名为 flickr.rating 和 yelp.rating, 两个数据集的每一行记录都包含三个字段, 以逗号分隔, 字段含义如下表 4.1、4.2 所示:

表 4.1 Flickr 数据集字段含义

名称	含义	类型	取值范围
user id	表示评分的用户 ID	int	/
photo id	表示被评分的照片 ID	int	/
rating	表示该用户对该照片的评分	int	[0,5]

表 4.2 Yelp 数据集字段含义

名称	含义	类型	取值范围
user id	表示评分的用户 ID	int	/
business id	表示被评分的商家 ID	int	/
stars	表示该用户对该商家的评分	int	[1,5]

为了让数据集适用于本实验的模型输入, 这里将 Flickr 数据集的 rating $\in [0,2]$ 的数据重新打标为 0, rating $\in [3,5]$ 的数据则重新打标为 1, 将 Yelp 数据集 stars $\in [1,2]$ 的数据重新打标为 0, stars $\in [3,5]$ 的数据则重新打标为 1。并均只保留标签为 1 的数据, 标签为 0 的数据可通过负样本采样得来。

预处理后, 数据变成 $\langle u, i, r \rangle$ 的三元组形式, 其中 $r \in \{0,1\}$, 一对正负样本则可形成偏序对, 可以输入模型。如: $\langle u, i, 1 \rangle$ 和 $\langle u, j, 0 \rangle$ 则形成偏序对 $\langle u, i, j \rangle$, 表示对于用户 u 而言, i 和 j 同时出现时更喜欢 i 。

另外两个下载到的数据集名为 item_vector 和 user_vector, 分别表示照片 (商家) 和用户的内容信息向量, 这里不进行预处理, 可直接输入模型。

2) 数据集划分

本实验按一定比例将预处理后的数据集 flickr.rating、yelp.rating 划分为训练集、验证集和测试集。使用训练集训练模型, 在验证集上评估模型的性能, 并调整模型的参数和超参数, 最后在测试集上评估模型的泛化能力, 得出模型的最终性能指标。划分前后的数据集样本数如下表 4.3、4.4 所示:

表 4.3 Flickr 各数据集样本数

数据集属性	数据集名称	数据集样本数
原始数据集	flickr.rating	327815
训练集	flickr.train.rating	282444
验证集	flickr.val.raing	13006
测试集	flickr.test.rating	32365

表 4.4 Yelp 各数据集样本数

数据集属性	数据集名称	数据集样本数
原始数据集	yelp.rating	207945
训练集	yelp.train.rating	185869
验证集	yelp.val.raing	3497
测试集	yelp.test.rating	18579

4.2 性能评测

4.2.1 评测方法

模型训练完成之后，能够得到用户、物品的隐向量综合表示，即模型中的 $\mathbf{u}$ 和 $\mathbf{v}$ 。评测时，需要先计算出 Top-K 推荐，即对每一个用户 u ，通过计算其与每一个物品 v 的分值，按从大到小的顺序排序并取出前 K 个做为 K 个推荐结果。本实验的 K 取值为 5，10 和 15。

对于热启动，用户和物品已经有了训练好的偏好向量表示 $\mathbf{u}_p$ 和 $\mathbf{v}_p$ ，此时隐向量综合表示可写为 $\mathbf{u} = \mathbf{u}_p + \hat{\mathbf{u}}_c, \mathbf{v} = \mathbf{v}_p + \hat{\mathbf{v}}_c$ 。而对于用户冷启动， $\mathbf{u}_p$ 应为 $\mathbf{0}$ ，此时隐向量综合表示写为 $\mathbf{u} = \mathbf{0} + \hat{\mathbf{u}}_c, \mathbf{v} = \mathbf{v}_p + \hat{\mathbf{v}}_c$ ，对于物品冷启动， $\mathbf{v}_p$ 应为 $\mathbf{0}$ ，隐向量综合表示写为 $\mathbf{u} = \mathbf{0} + \hat{\mathbf{u}}_c, \mathbf{v} = \mathbf{0} + \hat{\mathbf{v}}_c$ 。计算用户-物品分值如下式 4.1 所示：

$$score = \mathbf{u} \cdot \mathbf{v}^T \quad (4.1)$$

4.2.2 评测指标

评测指标的选择与模型性能有密切关系，通常情况下，评测指标能够准确地反映模型在实际应用中的性能表现。选择合适的评测指标对于评估模型性能和比较不同模型是非常重要的，不同的评测指标可以用来衡量模型在不同方面的表现，因此需要根据具体的任务目标选择适合的评价指标。

本实验选择的评测指标为命中率（Hits Ratio，简称 HR）^[38]和归一化折损累计增益（Normalized Discounted Cumulative Gain，简称 NDCG）^[39]。

HR@K 是指在 Top-K 个推荐结果中命中实际感兴趣物品的概率，计算公式如下式 4.2 所示：

$$HR@K = \frac{1}{N} \sum_{u \in U} hit(u) \quad (4.2)$$

其中， N 为用户数， U 为所有用户构成的集合。对于每个用户 u ，如果 Top-K 个推荐结果中存在实际感兴趣物品，则 $hit(u)$ 为 1，否则为 0。需要注意的是，HR 命中率只关注推荐列表中是否出现了用户真正感兴趣的物品，而不关心排名的顺序。因此，在使用 HR 命中率进行评估时，还需要结合其他指标来综合评价推荐系统的性能。

NDCG@K 不仅考虑了 Top-K 个推荐结果中是否包含实际感兴趣的物品，还考虑了这些物品出现的位置和相关程度，从而更全面地反映推荐系统的性能。在介绍 NDCG@K 之前，需要先介绍折损累计增益（Discounted Cumulative Gain，简称 DCG）和理想折损累计增益（Ideal Discounted Cumulative Gain，简称 IDCG），DCG@K 考虑了排序顺序的因素，使得排名靠前的物品增益更高，对排名靠后的物品增益进行折损，其计算公式为：

$$DCG@K = \sum_{i=1}^K \frac{rel(i)}{\log_2(i+1)} \quad (4.3)$$

其中 $rel(i)$ 表示物品 i 的相关性得分，这里取值为 0 或 1，IDCG@K 则是根据 $rel(i)$ 降序排列，即排列到最好状态算出来的 DCG@K。最后，NDCG@K 的具体计算公式如下公式 4.4，4.5 所示：

$$NDCG_u@K = \frac{DCG_u@K}{IDCG_u@K} \quad (4.4)$$

$$NDCG@K = \frac{1}{N} \sum_{u \in U} NDCG_u@K \quad (4.5)$$

其中， N 为用户数， U 为所有用户构成的集合。对于每一个用户 u ，根据实际感兴趣的物品构成的列表计算出一个 IDCG_u@K，再根据推荐系统的 Top-K 个物品构成的列表计算出一个 DCG_u@K，然后根据这两个值求出 NDCG_u@K，最后对所有用户求均值即可得到 NDCG@K。

4.3 模型超参数

模型的超参数是指在建立模型时需要设置的一些参数，这些参数对模型性能和效果有着重要的影响，而且通常不能从数据中直接学习得到，只能通过人为设置。下表 4.5 介绍了本实验模型中的各参数名称及其含义：

表 4.5 模型超参数名称及其含义

超参数名称	超参数含义
num_users	用户总数量
num_items	物品总数量
gpu_device	GPU 编号
data_name	数据集文件名
model_name	模型名称
dimension	u 和 v 的维度
learning_rate	学习率
epochs	模型训练轮数
num_negatives	训练时负样本采样数
num_evaluate	验证时负样本采样数
training_batch_size	训练时每个批次的正样本数
evaluate_batch_size	验证时每个批次的正样本数
num_layers	神经网络层数
topk	Top-K 中的 K
top5	/
top15	/

需要注意的是，为了保证训练时学习到更多样本的信息和验证时的真实性，训练和验证时都需要对样本进行负样本采样。预处理后的数据只有正样本，即 $\langle u, i, 1 \rangle$ 的形式，本实验设置的 num_negatives 参数表示训练时对于每一个正样本，再随机采样 num_negatives 个负样本，且每次训练都会更新负样本。num_evaluates 参数表示验证时对于每一个用户 u 采样 num_evaluates 个负样本，且每次只采样一次，负样本不再进行更新。

4.4 模型性能

4.4.1 实验方案

本文在 BPR 算法的基础上，引入了 DropoutNet 算法中的随机丢弃训练策略，使改进后的 BPR 算法（在此命名为 BPR_Dropout）既能保持原有热启动推荐的优秀表现，也能适用于冷启动推荐。

为了验证 BPR_Dropout 算法保留了良好的热启动性能，这里将 BPR_Dropout 与前文已经介绍的 BPR、PMF 进行对比来证明热启动推荐的表现。同时为了验证冷启动推荐的表现，通过使用不同超参数对 BPR_Dropout 算法的性能进行评测。

4.4.2 热启动性能

本文在 Flickr 数据集和 Yelp 数据集上对 BPR_Dropout、BPR、PMF 三个算法的热启动性能进行对比，设置的模型超参数如下表 4.6、4.7 所示。

表 4.6 Flickr 数据集实验超参数取值

超参数名称	超参数取值
num_users	8358
num_items	82120
gpu_device	1
data_name	flickr
model_name	bpr
dimension	64
learning_rate	1e-3
epochs	450
num_negatives	8
num_evaluate	1000
training_batch_size	512
evaluate_batch_size	2560
num_layers	1
topk	10
top5	5
top15	15

表 4.7 Yelp 数据集实验超参数取值

超参数名称	超参数取值
num_users	17237
num_items	38342
gpu_device	1
data_name	yelp
model_name	bpr
dimension	64
learning_rate	5e-4
epochs	450
num_negatives	8
num_evaluate	1000
training_batch_size	512
evaluate_batch_size	2560
num_layers	1
topk	10
top5	5
top15	15

三个模型在 Flickr 数据集上的 HR、NDCG 在 Top-5、Top-10、Top-15 上的表现如图 4.1 所示，可以看出，NDCG 总体上是小于 HR 的，这是因为在计算 NDCG 时还考虑了物品出现的位置和相关程度。

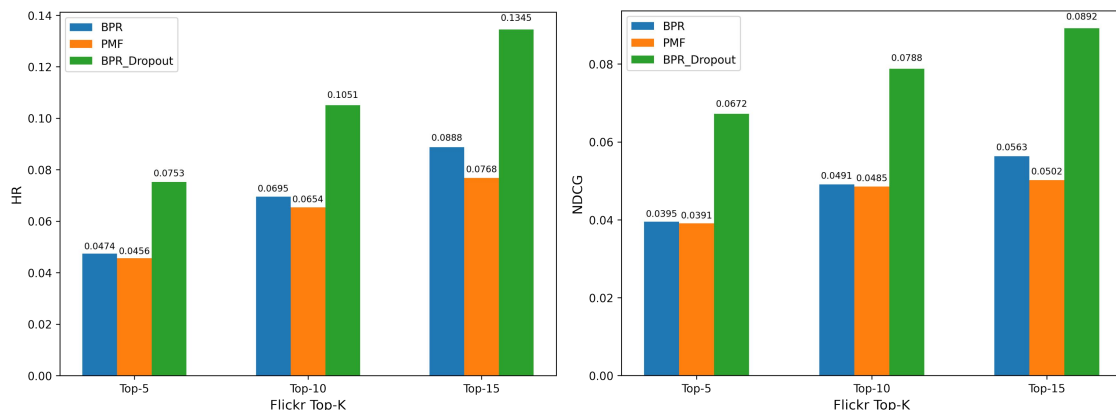


图 4.1 Flickr 数据集上三个模型的 HR 对比（左）、NDCG 对比（右）

三个模型在 Yelp 数据集上的 HR、NDCG 在 Top-5、Top-10、Top-15 上的表现如图 4.2 所示。

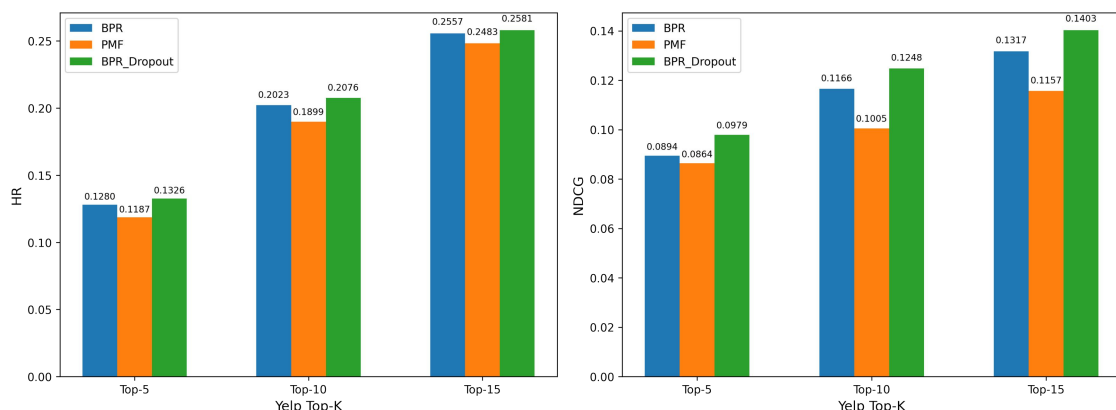


图 4.2 Yelp 数据集上三个模型的 HR 对比（左）、NDCG 对比（右）

从模型在两个不同数据集上的实验结果可以看出，对于 BPR、PMF 来说，两者都是基于矩阵分解的优秀算法，但 BPR 的性能不管是 HR 还是 NDCG 都略高于 PMF，这是因为 BPR 是基于用户对不同物品的偏好程度建模的，而 PMF 则是直接把没有交互的用户-物品对看作负样本，降低了模型性能。同时，对于本文所设计的 BPR_Dropout 而言，其热启动性能不仅没有降低反而有一定程度的上升，这是因为 BPR_Dropout 在三元组数据集上还引入了用户、物品的内容信息，增加了模型的学习能力。

4.4.3 冷启动性能

本实验通过使用不同超参数来观察其对冷启动推荐性能的影响，为了保证对比时的模型已经接近收敛，首先控制其他参数不变的情况下观察 epochs 对模型性能的影响。

因为 Top-K 在 K 取值为 5, 10 和 15 时的增减几乎是同步的，这里以 Top-10 为例，下图 4.3 和图 4.4 分别是 Flickr 数据集上 epochs 对用户、物品冷启动的 HR Top-10 和 NDCG Top-10 的影响。

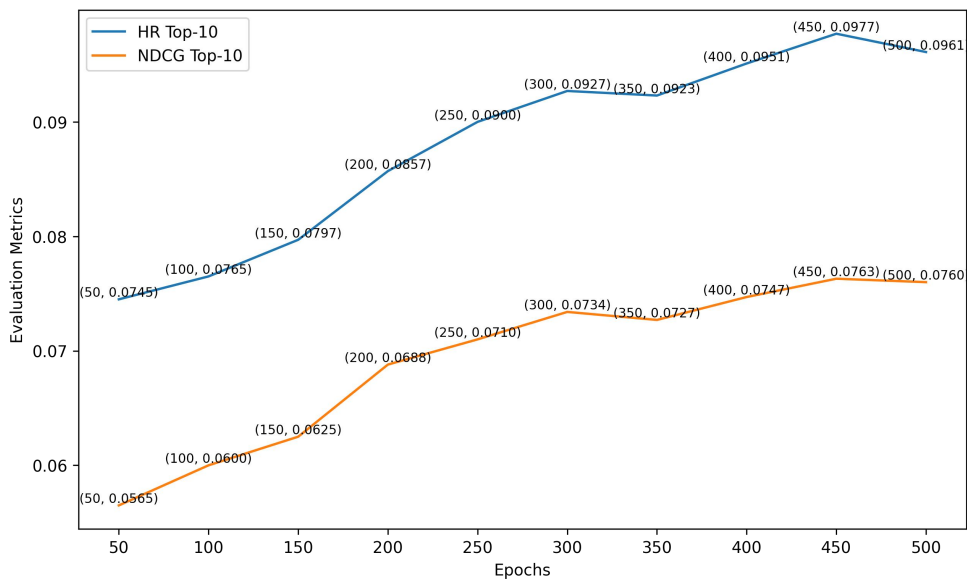


图 4.3 Flickr 数据集上 epochs 对用户冷启动性能的影响

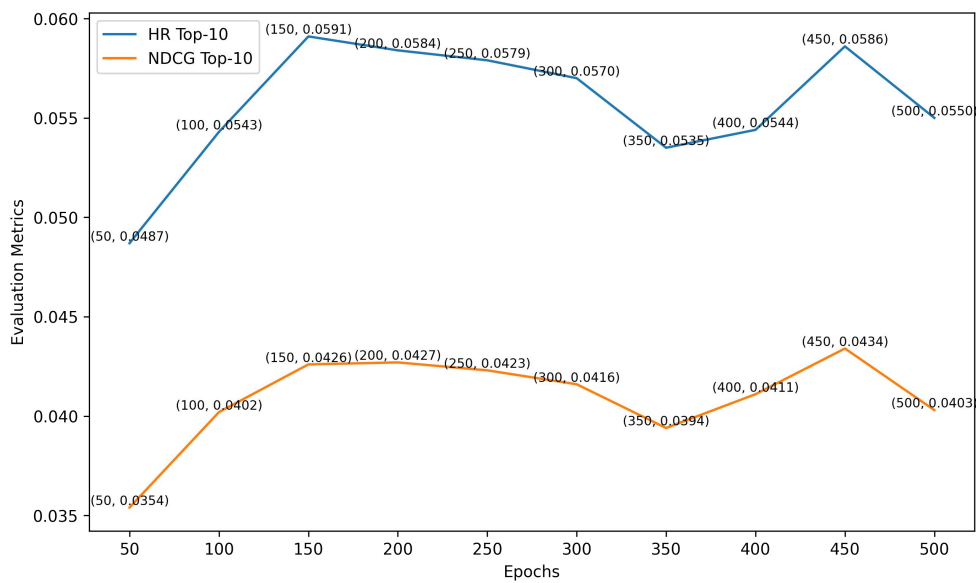


图 4.4 Flickr 数据集上 epochs 对物品冷启动性能的影响

从上图可以发现，对于用户冷启动，HR Top-10 和 NDCG Top-10 随着 epochs 的增长而增长，到 epochs=450 时增长开始减缓。而对于物品冷启动，HR Top-10 和 NDCG Top-10 随着 epochs 增长略微波动，但在 epochs=450 时，性能相对而言不错。对此，本实验折中选择 epochs=450 为 Flickr 数据集最终的实验参数。

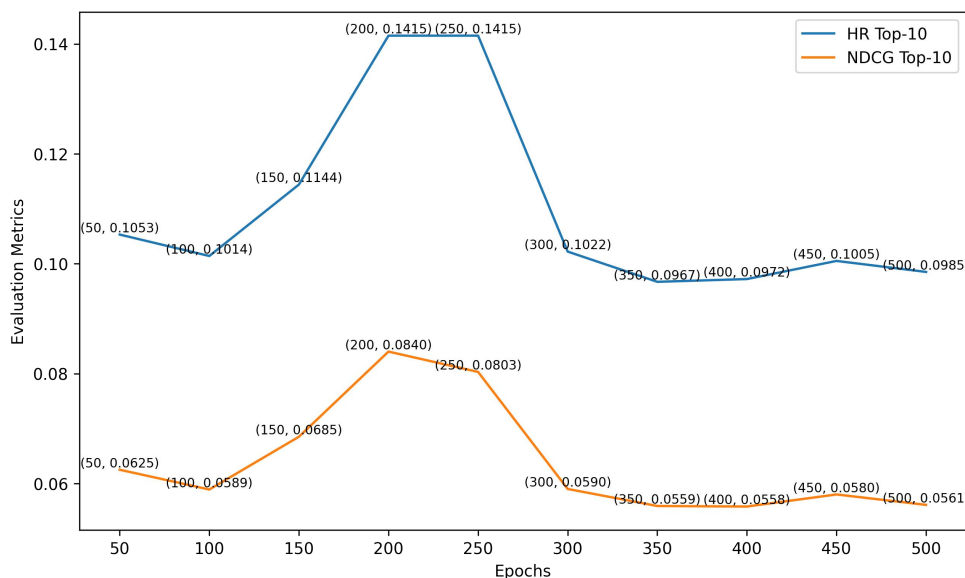


图 4.5 Yelp 数据集上 epochs 对用户冷启动性能的影响

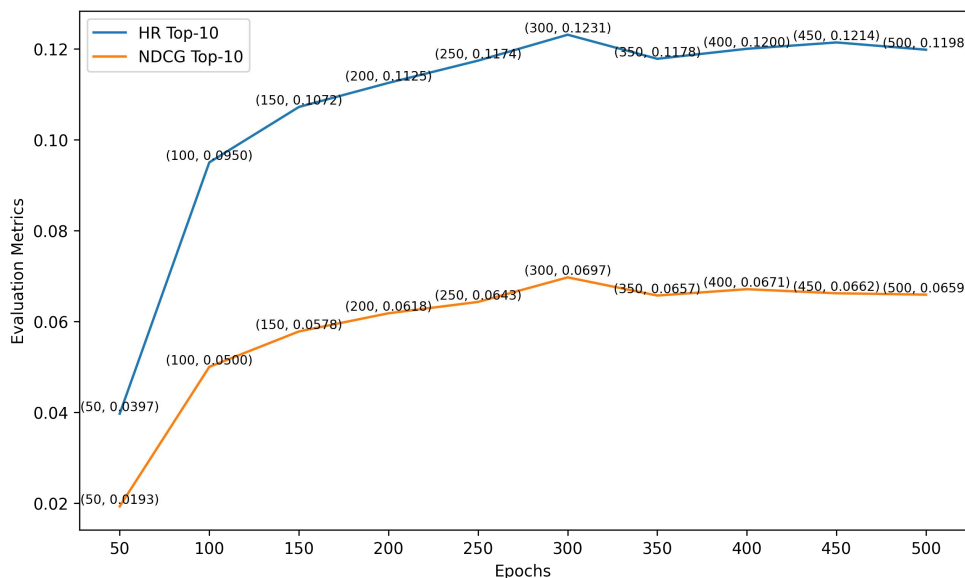


图 4.6 Yelp 数据集上 epochs 对物品冷启动性能的影响

上图 4.5 和图 4.6 分别是 Yelp 数据集上 epochs 对用户、物品冷启动的 HR Top-10 和 NDCG Top-10 的影响。本实验折中选择 epochs=250 为 Yelp 数据集最终的实验参数。

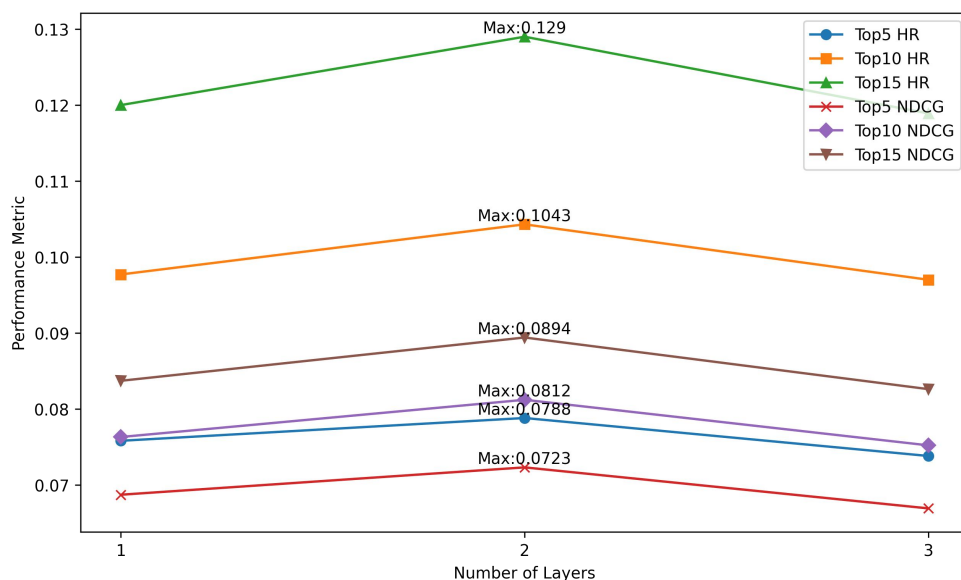


图 4.7 Flickr 数据集上神经网络层数对用户冷启动的影响

在用户、物品内容信息向量上连接不同层数的神经网络会对模型的冷启动性能产生不同的影响，下表 4.8 为 Flickr 数据集上神经网络层数对用户冷启动性能的影响，其对应的折线图如图 4.7 所示。

表 4.8 Flickr 数据集上神经网络层数对用户冷启动的影响

	1 layer		2 layers		3 layers	
	HR	NDCG	HR	NDCG	HR	NDCG
Top-5	0.0758	0.0687	0.0788	0.0723	0.0738	0.0669
Top-10	0.0977	0.0763	0.1043	0.0812	0.0970	0.0752
Top-15	0.1200	0.0837	0.1290	0.0894	0.1189	0.0826

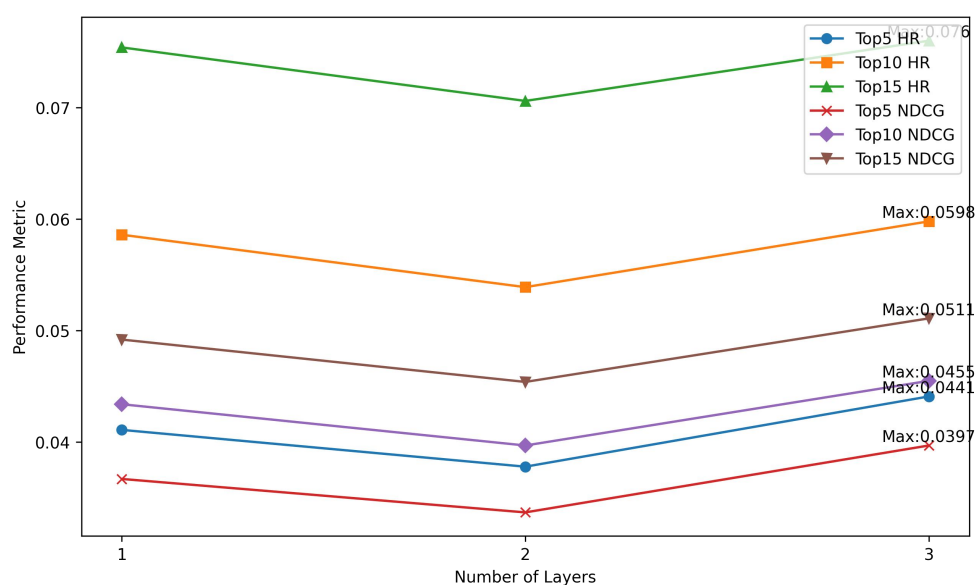


图 4.8 Flickr 数据集上神经网络层数对物品冷启动的影响

下表 4.9 为 Flickr 数据集上神经网络层数对物品冷启动性能的影响，上图 4.8 为其所对应的折线图。

表 4.9 Flickr 数据集上神经网络层数对物品冷启动的影响

	1 layer		2 layers		3 layers	
	HR	NDCG	HR	NDCG	HR	NDCG
Top-5	0.0411	0.0367	0.0378	0.0337	0.0441	0.0397
Top-10	0.0586	0.0434	0.0539	0.0397	0.0598	0.0455
Top-15	0.0754	0.0492	0.0706	0.0454	0.0760	0.0511

下表 4.10 为 Yelp 数据集上神经网络层数对用户冷启动性能的影响，其对应的折线图如图 4.9 所示。

表 4.10 Yelp 数据集上神经网络层数对用户冷启动的影响

	1 layer		2 layers		3 layers	
	HR	NDCG	HR	NDCG	HR	NDCG
Top-5	0.0872	0.0640	0.1001	0.0726	0.0994	0.0734
Top-10	0.1415	0.0803	0.1553	0.0923	0.1550	0.0931
Top-15	0.1693	0.0911	0.2022	0.1062	0.1947	0.1051

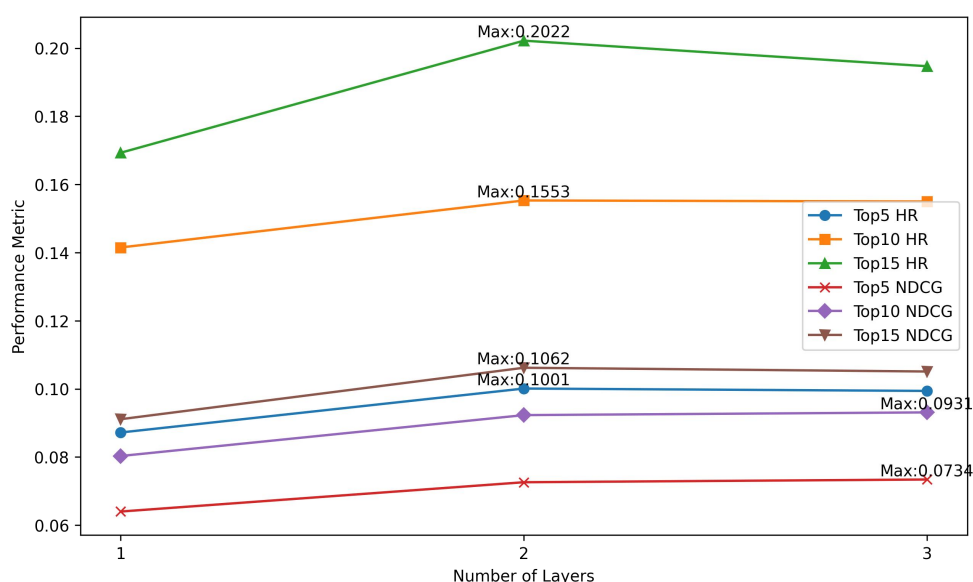


图 4.9 Yelp 数据集上神经网络层数对用户冷启动的影响

下表 4.11 为 Yelp 数据集上神经网络层数对物品冷启动性能的影响，其对应的折线图如图 4.10 所示。

表 4.11 Yelp 数据集上神经网络层数对物品冷启动的影响

	1 layer		2 layers		3 layers	
	HR	NDCG	HR	NDCG	HR	NDCG
Top-5	0.0661	0.0456	0.1017	0.0747	0.1007	0.0730
Top-10	0.1174	0.0643	0.1632	0.0970	0.1609	0.0946
Top-15	0.1582	0.0766	0.2119	0.1117	0.2075	0.1087

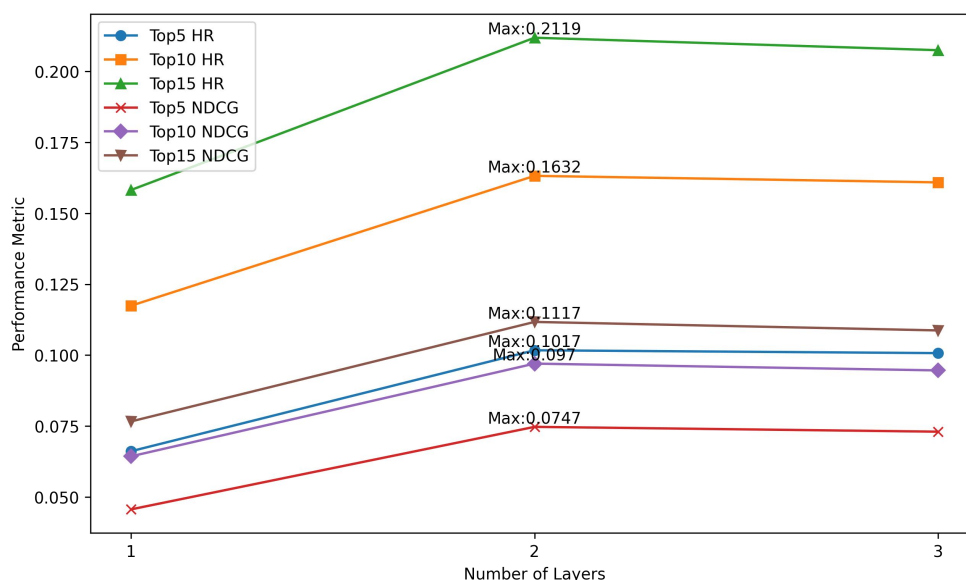


图 4.10 Yelp 数据集上神经网络层数对物品冷启动的影响

从上图可以得出三个结论：一是本算法在两个数据集上的冷启动场景中都取得了不错的效果，可以确定本算法在冷启动推荐中的有效性。二是模型的性能并不是神经网络层数越多越好，层数多了反而可能带来负面影响，这可能是由于模型过拟合导致的。三是在 Flickr 数据集上神经网络层数的多少对于用户冷启动和物品冷启动的影响是不同的，用户冷启动的最佳神经网络层数为 2，这时所有指标均为最高，而物品冷启动的最佳神经网络层数则是 3。而在 Yelp 数据集上神经网络层数的多少对于用户冷启动和物品冷启动的影响是几乎相同的，都是神经网络层数为 2 时效果最佳。基于此，若要将此实验中的方法部署于业界的某些数据集上，可以训练两个模型，分别用于用户冷启动和物品冷启动。

4.5 本章小结

本章主要从实验数据介绍、性能评测方法、模型超参数、模型性能评估等方面对本文所设计的冷启动推荐算法的性能进行了测试，并最终表明本文所设计的算法不仅保持了 BPR 在热启动推荐中的优秀表现还可以适用于冷启动推荐。

第5章 总结与展望

5.1 主要工作

本文主要研究了面向冷启动问题的内容推荐算法，以解决传统推荐算法在新用户或新物品上的表现不佳的问题。通过在 BPR 的基础上采用 DropoutNet 的随机丢弃策略，让 BPR 在保持原有热启动性能的同时还能适用于冷启动。

本文首先对推荐系统和冷启动推荐的发展现状进行了介绍，建立了对本文研究主题的背景和意义的认识。接着介绍了实验所涉及的基础知识，包括推荐系统基本知识和深度学习基础知识，为后续算法的设计打下基础。然后详细介绍了本文所设计的面向冷启动问题的内容推荐算法，从问题描述、算法设计、模型训练三个方面进行了阐述。最后，在模型性能评估和实验结果分析方面，对本文所设计算法的性能表现进行了详细的测试和分析。

实验结果表明，本文所设计的算法不仅在热启动场景下保持了 BPR 的优异表现，而且在冷启动场景下也取得了较好的表现。因此，本文算法具有很好的应用前景，可以为推荐系统提供更好的服务质量和更良好的用户体验。

5.2 后续工作

本文还有很多可以进一步深入探索的方向，首先，在设计推荐算法时采用了 DropoutNet 中的随机丢弃策略，但在丢弃方法上仅使用了简单的置 0 操作。为了提升模型冷启动推荐性能，可以尝试将置 0 操作改为用户或物品偏好向量的平均值。此外，可以采用更多的数据集进行实验，以验证本文算法的普适性和鲁棒性。

另外，在算法设计方面，可以探索如何引入注意力机制^[40]来对不同的物品进行加权，使得推荐结果更加个性化和符合用户兴趣。同时，模型的计算效率也是一个关键问题，可以探索如何使用分布式计算或 GPU 加速等技术来提高模型的训练速度和推荐效率。最后，在应用场景中，还可以考虑如何将本文所提出的算法应用于现实生活中的推荐服务中，从而为用户提供更好的推荐体验。

参考文献

- [1] Ko H, Lee S, Park Y, et al. A survey of recommendation systems: recommendation models, techniques, and application fields[J]. Electronics, 2022, 11(1): 141-189.
- [2] 史海燕, 倪云瑞. 推荐系统冷启动问题研究进展[J]. 图书馆学研究, 2021, 12(1): 2-10.
- [3] Volkovs M, Yu G, Poutanen T. Dropoutnet: Addressing cold start in recommender systems[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc. , 2017: 4964-4973.
- [4] 于蒙, 何文涛, 周绪川, 等. 推荐系统综述[J]. 计算机应用, 2022, 42(6): 1898-1913.
- [5] Melville P, Mooney R J, Nagarajan R. Content-boosted collaborative filtering for improved recommendations[J]. Association for the Advancement of Artificial Intelligence, 2002, 23(1): 187-192.
- [6] Goldberg D, Nichols D, Oki B M, et al. Using collaborative filtering to weave an information tapestry[J]. Communications of the ACM, 1992, 35(12): 61-70.
- [7] Lee D D, Seung H S. Learning the parts of objects by non-negative matrix factorization[J]. Nature, 1999, 401(6755): 788-791.
- [8] Mnih A, Salakhutdinov R R. Probabilistic matrix factorization[C]//Proceedings of the 20th International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc. , 2007: 1257-1264.
- [9] Bell R, Koren Y, Volinsky C. Modeling relationships at multiple scales to improve accuracy of large recommender systems[C]//Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining. New York, NY, USA: Association for Computing Machinery, 2007: 95-104.
- [10] Burke R. Hybrid recommender systems: Survey and experiments[J]. User modeling and user-adapted interaction, 2002, 12(4): 331-370.
- [11] Cheng H T, Koc L, Harmsen J, et al. Wide & deep learning for recommender systems[C]//Proceedings of the 1st workshop on deep learning for recommender systems. New York, NY, USA: Association for Computing Machinery, 2016: 7-10.

- [12] Rumelhart D E, Hinton G E, Williams R J. Learning representations by back-propagating errors[J]. *Nature*, 1986, 323(6088): 533-536.
- [13] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. *Proceedings of the IEEE*, 1998, 86(11): 2278-2324.
- [14] Scarselli F, Gori M, Tsoi A C, et al. The graph neural network model[J]. *IEEE Transactions on Neural Networks*, 2008, 20(1): 61-80.
- [15] Cai D, Qian S, Fang Q, et al. User cold-start recommendation via inductive heterogeneous graph neural network[J]. *ACM Transactions on Information Systems*, 2023, 41(3): 1-27.
- [16] Panda D K, Ray S. Approaches and algorithms to mitigate cold start problems in recommender systems: a systematic literature review[J]. *Journal of Intelligent Information Systems*, 2022, 59(2): 341-366.
- [17] Pan F, Li S, Ao X, et al. Warm up cold-start advertisements: Improving ctr predictions via learning to learn id embeddings[C]//*Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. New York, NY, USA: Association for Computing Machinery, 2019: 695-704.
- [18] Wang H, Zhang F, Zhang M, et al. Knowledge-aware graph neural networks with label smoothness regularization for recommender systems[C]//*Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. New York, NY, USA: Association for Computing Machinery, 2019: 968-977.
- [19] 赵晔辉, 柳林, 王海龙, 等. 知识图谱推荐系统研究综述[J]. *计算机科学与探索*, 2023, 17(4): 771-791.
- [20] Lu Y, Fang Y, Shi C. Meta-learning on heterogeneous information networks for cold-start recommendation[C]//*Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. New York, NY, USA: Association for Computing Machinery, 2020: 1563-1573.
- [21] 朱冬亮, 文奕, 万子琛. 基于知识图谱的推荐系统研究综述[J]. *数据分析与知识发现*, 2022, 5(12): 1-13.
- [22] Wei Y, Wang X, Li Q, et al. Contrastive learning for cold-start recommendation[C]//*Proceedings of the 29th ACM International Conference on*

- Multimedia. New York, NY, USA: Association for Computing Machinery, 2021: 5382-5390.
- [23] Natarajan S, Vairavasundaram S, Natarajan S, et al. Resolving data sparsity and cold start problem in collaborative filtering recommender system using linked open data[J]. Expert Systems with Applications, 2020, 149(4): 113-248.
- [24] 方阳, 谭真, 陈子阳, 等. 用于冷启动推荐的异质信息网络对比元学习[J]. 软件学报, 2023, 34(10): 1-19.
- [25] El-Kishky A, Markovich T, Park S, et al. Twihin: Embedding the twitter heterogeneous information network for personalized recommendation[C]//Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. New York, NY, USA: Association for Computing Machinery, 2022: 2842-2850.
- [26] Salton G, Buckley C. Term-weighting approaches in automatic text retrieval[J]. Information Processing & Management, 1988, 24(5): 513-523.
- [27] Church K W. Word2Vec[J]. Natural Language Engineering, 2017, 23(1): 155-162.
- [28] Bishop C M. Neural networks for pattern recognition[M]. Oxford University Press, 1995.
- [29] Polson N G, Scott J G. On the half-Cauchy prior for a global scale parameter[J]. Bayesian Analysis, 2012, 7(4): 887-902.
- [30] Nair V, Hinton G E. Rectified linear units improve restricted boltzmann machines[C]//Proceedings of the 27th International Conference on machine learning. Madison, WI, USA: Omnipress, 2010: 807-814.
- [31] Rosenblatt F. The perceptron: a probabilistic model for information storage and organization in the brain[J]. Psychological review, 1958, 65(6): 386-408.
- [32] Schmidt M, Le R N, Bach F. Minimizing finite sums with the stochastic average gradient[J]. Mathematical Programming, 2017, 162(1): 83-112.
- [33] Lo A W, MacKinlay A C. Stock market prices do not follow random walks: Evidence from a simple specification test[J]. The review of financial studies, 1988, 1(1): 41-66.
- [34] 梁云. TensorFlow 实战 Google 深度学习框架[M]. 人民邮电出版社, 2018.
- [35] Rendle S, Freudenthaler C, Gantner Z, et al. BPR: Bayesian personalized ranking from implicit feedback[C]//Proceedings of the Twenty-Fifth Conference on

- Uncertainty in Artificial Intelligence. Arlington, Virginia, USA: AUAI Press, 2012: 452–461.
- [36] Srivastava N, Hinton G, Krizhevsky A, et al. Dropout: a simple way to prevent neural networks from overfitting[J]. The journal of machine learning research, 2014, 15(1): 1929-1958.
- [37] Zhang S, Yao L, Sun A, et al. Deep learning based recommender system: A survey and new perspectives[J]. ACM computing surveys (CSUR), 2019, 52(1): 1-38.
- [38] Koren Y. Factorization meets the neighborhood: a multifaceted collaborative filtering model[C]//Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining. New York, NY, USA: Association for Computing Machinery, 2008: 426-434.
- [39] Järvelin K, Kekäläinen J. Cumulated gain-based evaluation of IR techniques[J]. ACM Transactions on Information Systems, 2002, 20(4): 422-446.
- [40] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]// Proceedings of the 31st International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc. , 2017: 6000–6010.

致谢

行文于此，落笔为终，大学四年生活也将画上句号。品味一路走来的点滴，我心怀无尽感恩。

首先，我要特别感谢我的导师黎俊伟老师，他在我撰写论文过程中给予了我非常宝贵的建议和指导，他的专业知识和悉心指导是我完成论文的关键所在。

其次，我要感谢为我提供帮助、支持和鼓舞的老师、同学和学长学姐们，感谢他们的帮助让我不断提升。

最后，我要感谢我的家人朋友们，是他们让我深深感受到生活的美好，感谢他们一直以来给予我的理解、鼓励和支持。

附录 A 部分源代码

A.1 DropoutNet 核心源代码

```

while d_train.terminal_flag:
    dropout_way = random.random()
    d_train.getTrainRankingBatch()
    d_train.linkedMap()
    # set_trace()
    train_feed_dict = {}
    # user 冷启动 置为 0
    if dropout_way < 0.2:
        for i, (key, value) in enumerate(model.map_dict['train'].items()):
            if i == 0:
                train_feed_dict[key] = np.reshape(len(d_train.data_dict[value]) *
[conf.num_users], [-1, 1])
            else:
                train_feed_dict[key] = d_train.data_dict[value]
        # item 冷启动 置为 0
    if dropout_way >= 0.2 and dropout_way < 0.4:
        for i, (key, value) in enumerate(model.map_dict['train'].items()):
            if i == 2:
                train_feed_dict[key] = np.reshape(len(d_train.data_dict[value]) *
[conf.num_items], [-1, 1])
            else:
                train_feed_dict[key] = d_train.data_dict[value]
        # user、item 冷启动 置为 0
    if dropout_way >= 0.4 and dropout_way < 0.5:
        for i, (key, value) in enumerate(model.map_dict['train'].items()):
            if i == 0:
                train_feed_dict[key] = np.reshape(len(d_train.data_dict[value]) *
[conf.num_users], [-1, 1])
            if i == 2:
                train_feed_dict[key] = np.reshape(len(d_train.data_dict[value]) *
[conf.num_items], [-1, 1])
            else:
                train_feed_dict[key] = d_train.data_dict[value]
        # 热启动
    if dropout_way >= 0.5:
        for i, (key, value) in enumerate(model.map_dict['train'].items()):
            train_feed_dict[key] = d_train.data_dict[value]

```

A.2 BPR 核心源代码

```

# BPR
latest_user_latent = tf.gather_nd(self.user_embedding, self.user_input)

```

```
latest_item_latent = tf.gather_nd(self.item_embedding, self.item_input)

# content embedding
latest_user_content = tf.gather_nd(self.user_content_embedding,
self.user_content_input)
latest_item_content = tf.gather_nd(self.item_content_embedding,
self.item_content_input)
# 全连接层 256->256
latest_user_content = tf.layers.dense(inputs=latest_user_content, units=256,
activation=tf.nn.relu)
latest_item_content = tf.layers.dense(inputs=latest_item_content, units=256,
activation=tf.nn.relu)

# 全连接层 256->64
latest_user_content = tf.layers.dense(inputs=latest_user_content,
units=self.conf.dimension, activation=tf.nn.relu)
latest_item_content = tf.layers.dense(inputs=latest_item_content,
units=self.conf.dimension, activation=tf.nn.relu)

latest_user = tf.add(latest_user_content, latest_user_latent)
latest_item = tf.add(latest_item_content, latest_item_latent)

# tf.multiply 是 TensorFlow 中的一个用于对两个张量进行 逐元素 相乘的函数
self.predict_vector = tf.multiply(latest_user, latest_item)
self.prediction = tf.sigmoid(tf.reduce_sum(self.predict_vector, 1, keepdims=True))

# 热启动 loss
self.loss = tf.nn.l2_loss(self.labels_input - self.prediction)
# user 冷启动 loss
self.user_cold_predict_vector = tf.multiply(latest_user_content, latest_item)
self.user_cold_prediction = tf.sigmoid(tf.reduce_sum(self.user_cold_predict_vector, 1,
keepdims=True))
self.user_cold_loss = tf.nn.l2_loss(self.labels_input - self.user_cold_prediction)
# item 冷启动 loss
self.item_cold_predict_vector = tf.multiply(latest_user, latest_item_content)
self.item_cold_prediction = tf.sigmoid(tf.reduce_sum(self.item_cold_predict_vector, 1,
keepdims=True))
self.item_cold_loss = tf.nn.l2_loss(self.labels_input - self.item_cold_prediction)
# user、item 冷启动
self.user_item_cold_predict_vector = tf.multiply(latest_user_content,
latest_item_content)
self.user_item_cold_prediction =
tf.sigmoid(tf.reduce_sum(self.user_item_cold_predict_vector, 1, keepdims=True))
self.user_item_cold_loss = tf.nn.l2_loss(self.labels_input -
self.user_item_cold_prediction)

self.opt_loss = tf.nn.l2_loss(self.labels_input - self.prediction)
self.opt = tf.train.AdamOptimizer(self.conf.learning_rate).minimize(self.opt_loss)
self.init = tf.global_variables_initializer()
```

A.3 Python 可视化部分源代码

```
import matplotlib.pyplot as plt
import numpy as np

# 定义评测指标结果
BPR = [0.0395, 0.0491, 0.0563]
PMF = [0.0391, 0.0485, 0.0502]
BPR_dropout = [0.0672, 0.0788, 0.0892]
ind = np.arange(3)
width = 0.2

# 绘制柱状图
fig, ax = plt.subplots(figsize=(8, 6), dpi=300)
rects1 = ax.bar(ind - width, BPR, width, label='BPR')
rects2 = ax.bar(ind, PMF, width, label='PMF')
rects3 = ax.bar(ind + width, BPR_dropout, width, label='BPR_Dropout')

# 在柱状图顶部添加数字标注
for rect in rects1+rects2+rects3:
    height = rect.get_height()
    ax.text(rect.get_x() + rect.get_width()/2., 1.02*height, '%.4f' % float(height),
            ha='center', va='bottom', fontsize=8)

# 添加图例、坐标轴标签等
ax.set_xlabel('Flickr Top-K', fontsize=12)
ax.set_ylabel('NDCG', fontsize=12)
ax.set_xticks(ind)
ax.set_xticklabels(('Top-5', 'Top-10', 'Top-15'), fontsize=10)
ax.legend(fontsize=10)

# 设置坐标轴自适应范围
ax.autoscale_view()

# 显示并保存图像
plt.show()
plt.savefig('my_plot.png', dpi=300, bbox_inches='tight')
```


附录 B 英文原文

DropoutNet: Addressing Cold Start in Recommender Systems

论文原文: <https://proceedings.neurips.cc/paper/2017/hash/dbd22ba3bd0df8f385bdac3e9f8be207-Abstract.html>

英文 Abstract

Latent models have become the default choice for recommender systems due to their performance and scalability. However, research in this area has primarily focused on modeling user-item interactions, and few latent models have been developed for cold start. Deep learning has recently achieved remarkable success showing excellent results for diverse input types. Inspired by these results we propose a neural network based latent model called DropoutNet to address the cold start problem in recommender systems. Unlike existing approaches that incorporate additional content-based objective terms, we instead focus on the optimization and show that neural network models can be explicitly trained for cold start through dropout. Our model can be applied on top of any existing latent model effectively providing cold start capabilities, and full power of deep architectures. Empirically we demonstrate state-of-the-art accuracy on publicly available benchmarks. Code is available at <https://github.com/layer6ai-labs/DropoutNet>.

B.1 Introduction

Popularity of online content delivery services, e-commerce, and social web has highlighted an important challenge of surfacing relevant content to consumers. Recommender systems have proven to be effective tools for this task, receiving increasingly more attention. One common approach to building accurate recommender models is collaborative filtering (CF). CF is a method of making predictions about an individual's preferences based on the preference information from other users. CF has been shown to work well across various domains [19], and many successful web-services such as Netflix, Amazon and YouTube use CF to deliver highly personalized recommendations to their users.

The majority of the existing approaches in CF can be divided into two categories: neighbor-based and model-based. Model-based approaches, and in particular latent models, are typically the preferred choice since they build compact representations of

the data and achieve high accuracy. These representations are optimized for fast retrieval and can be scaled to handle millions of users in real-time. For these reasons we concentrate on latent approaches in this work. Latent models are typically learned by applying a variant of low rank approximation to the target preference matrix. As such, they work well when lots of preference information is available but start to degrade in highly sparse settings. The most extreme case of sparsity known as *cold start* occurs when no preference information is available for a given user or item. In such cases, the only way a personalized recommendation can be generated is by incorporating additional content information. Base latent approaches cannot incorporate content, so a number of hybrid models have been proposed [3, 21, 22] to combine preference and content information. However, most hybrid methods introduce additional objective terms considerably complicating learning and inference. Moreover, the content part of the objective is typically generative [21, 9, 22] forcing the model to “explain” the content rather than use it to maximize recommendation accuracy.

Recently, deep learning has achieved remarkable success in areas such as computer vision [15, 11], speech [12, 10] and natural language processing [5, 16]. In all of these areas end-to-end deep neural network (DNN) models achieve state-of-the-art accuracy with virtually no feature engineering. These results suggest that deep learning should also be highly effective at modeling content for recommender systems. However, while there has been some recent progress in applying deep learning to CF [7, 22, 6, 23], little investigation has been done on using deep learning to address the cold start problem.

In this work we propose a model to address this gap. Our approach is based on the observation that cold start is equivalent to the missing data problem where preference information is missing. Hence, instead of adding additional objective terms to model content, we modify the learning procedure to explicitly condition the model for the missing input. The key idea behind our approach is that by applying dropout [18] to input mini-batches, we can train DNNs to generalize to missing input. By selecting an appropriate amount of dropout we show that it is possible to learn a DNN-based latent model that performs comparably to state-of-the-art on warm start while significantly outperforming it on cold start. The resulting model is simpler than most hybrid approaches and uses a single objective function, jointly optimizing all components to maximize recommendation accuracy.

An additional advantage of our approach is that it can be applied on top of any

existing latent model to provide/enhance its cold start capability. This requires virtually no modification to the original model thus minimizing the implementation barrier for any production environment that's already running latent models. In the following sections we give a detailed description of our approach and show empirical results on publicly available benchmarks.

B.2 Framework

In a typical CF problem we have a set of N users $U = \{u_1, \dots, u_N\}$ and a set of M items $V = \{v_1, \dots, v_M\}$. The users' feedback for the items can be represented by an $N \times M$ preference matrix $\mathbf{R}$ where $\mathbf{R}_{uv}$ is the preference for item v by user u . $\mathbf{R}_{uv}$ can be either explicitly provided by the user in the form of rating, like/dislike etc., or inferred from implicit interactions such as views, plays and purchases. In the explicit setting $\mathbf{R}$ typically contains graded relevance (e.g., 1-5 ratings), while in the implicit setting $\mathbf{R}$ is often binary; we consider both cases in this work. When no preference information is available $\mathbf{R}_{uv} = 0$. We use $U(v) = \{u \in U \mid \mathbf{R}_{uv} \neq 0\}$ to denote the set of users that expressed preference for v , and $V(u) = \{v \in V \mid \mathbf{R}_{uv} \neq 0\}$ to denote the set of items that u expressed preference for. In cold start no preference information is available and we formally define cold start when $V(u) = \emptyset$ and $U(v) = \emptyset$ for a given user u and item v .

Additionally, in many domains we often have access to content information for both users and items. For items, this information can come in the form of text, audio or images/video. For users we could have profile information (age, gender, location, device etc.), and social media data (Facebook, Twitter etc.). This data can provide highly useful signal for recommender models, and is particularly effective in sparse and cold start settings where little or no preference information is available. After applying relevant transformations most content information can be represented by fixed-length feature vectors. We use Φ^U and Φ^V to denote the content features for users and items respectively where Φ_u^U (Φ_v^V) is the content feature vector for user u (item v). When content is missing the corresponding feature vector is set to 0. The goal is to use the preference information $\mathbf{R}$ together with content Φ^U and Φ^V , to learn accurate and robust recommendation model. Ideally this model should handle all stages of the

user/item journey: from cold start, to early stage sparse preferences, to a late stage well-defined preference profile.

B.3 Relevant Work

A number of hybrid latent approaches have been proposed to address cold start in CF. One of the more popular models is the collaborative topic regression (CTR) [21] which combines latent Dirichlet allocation (LDA) [4] and weighted matrix factorization (WMF) [13]. CTR interpolates between LDA representations in cold start and WMF when preferences are available. Recently, several related approaches have been proposed. Collaborative topic Poisson factorization (CTPF) [8] uses a similar interpolation architecture but replaces both LDA and WMF components with Poisson factorization [9]. Collaborative deep learning (CDL) [22] is another approach with analogous architecture where LDA is replaced with a stacked denoising autoencoder [20].

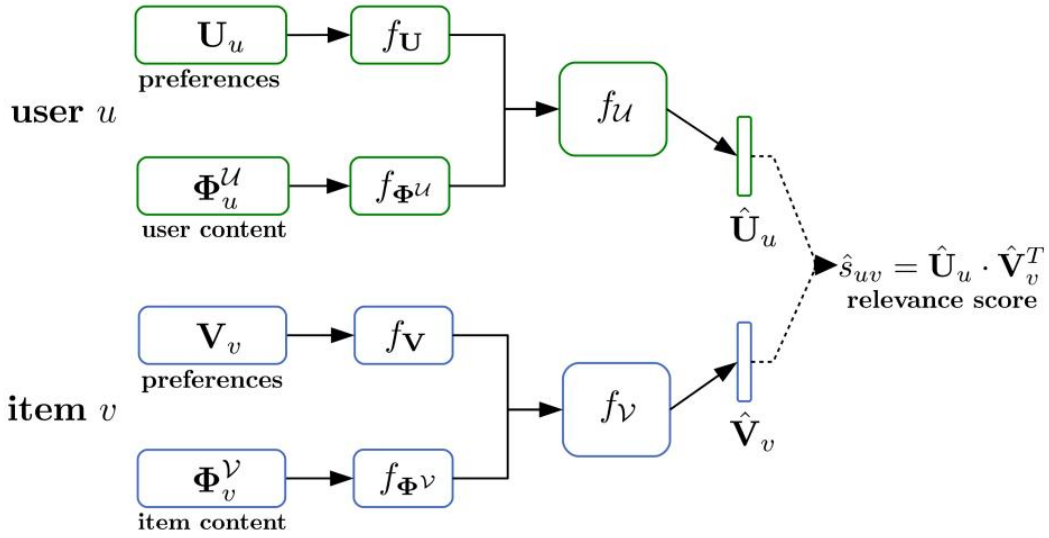


Figure 1: DropoutNet architecture diagram. For each user u , the preference $\mathbf{U}_u$ and content $\Phi_u^{\mathcal{U}}$ inputs are first passed through the corresponding DNNs $f_{\mathbf{U}}$ and $f_{\Phi^{\mathcal{U}}}$. Top layer activations are then concatenated together and passed to the fine-tuning network $f_{\mathcal{U}}$ which outputs the latent representation $\hat{\mathbf{U}}_u$. Items are handled in a similar fashion with $f_{\mathbf{V}}$, $f_{\Phi^{\mathcal{V}}}$ and $f_{\mathcal{V}}$ to produce $\hat{\mathbf{V}}_v$. All components are optimized jointly with back-propagation and then kept fixed during inference. Retrieval is done in the new latent space using $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ that replace the original representations $\mathbf{U}$ and $\mathbf{V}$.

While these models achieve highly competitive performance, they also share several disadvantages. First, they incorporate both preference and content components into the objective function making it highly complex. CDL for example, contains four objective terms and requires tuning three combining weights in addition to WMF and autoencoder parameters. This makes it challenging to tune these models on large datasets where every parameter setting experiment is expensive and time consuming. Second, the formulation of each model assumes cold start items and is not applicable to cold start users. Most online services have to frequently incorporate new users and items and thus require models that can handle both. In principle it is possible to derive an analogous model for users and jointly optimize both models. However, this would require an even more complex objective nearly doubling the number of free parameters. One of the main questions that we aim to address with this work is whether we develop a simpler cold start model that is applicable to both users and items?

In addition to CDL, a number of approaches haven been proposed to leverage DNNs for CF. One of the earlier approaches DeepMusic [7] aimed to predict latent representations learned by a latent model using content only DNN. Recently, [6] described YouTube’s two-stage recommendation model that takes as input user session (recent plays and searches) and profile information. Latent representations for items in a given session are averaged, concatenated with profile information, and passed to a DNN which outputs a session-dependent latent representation for the user. Averaging the items addresses variable length input problem but can loose temporal aspects of the session. To more accurately model how users’ preferences change over time a recurrent neural network (RNN) approach has been proposed by [23]. RNN is applied sequentially to one item at a time, and after all items are processed hidden layer activations are used as latent representation.

Many of these models show clear benefits of applying deep architectures to CF. However, few investigate cold start and sparse setting performance when content information is available. Arguably, we expect deep learning to be the most beneficial in these scenarios due to its excellent generalization to various content types. Our proposed approach aims to leverage this advantage and is most similar to [6]. We also use latent representations as preference feature input for users and items, and combine them with content to train a hybrid DNN-based model. But unlike [6] which focuses primarily on warm start users, we develop analogous models for both users and items,

and then show how these models can be trained to explicitly handle cold start.

B.4 Our Approach

In this section we describe the architecture of our model that we call DropoutNet, together with learning and inference procedures. We begin with input representation. Our aim is to develop a model that is able to handle both cold and warm start scenarios. Consequently, input to the model needs to contain content and preference information. One option is to directly use rows and columns of $\mathbf{R}$ in their raw form. However, these become prohibitively large as the number of users and items grows. Instead, we take a similar approach to [6] and [23], and use latent representations as preference input. Latent models typically approximate the preference matrix with a product of low rank matrices $\mathbf{U}$ and $\mathbf{V}$:

$$\mathbf{R}_{uv} \approx \mathbf{U}_u \mathbf{V}_v^T$$

where $\mathbf{U}_u$ and $\mathbf{V}_v$ are the latent representations for user u and item v respectively. Both $\mathbf{U}$ and $\mathbf{V}$ are dense and low dimensional with rank $D \ll \min(M, N)$. Noting the strong performance of latent approaches on a wide range of CF datasets, it is adequate to assume that the latent representations accurately summarize preference information about users and items. Moreover, low input dimensionality significantly reduces model complexity for DNNs since activation size of the first hidden layer is directly proportional to the input size. Given these advantages we set the input to $[\mathbf{U}_u, \Phi_u^U]$ and $[\mathbf{V}_v, \Phi_v^V]$ for each user u and item v respectively.

B.4.1 Model Architecture

Given the joint preference-content input we propose to apply a DNN model to map it into a new latent space that incorporates both content and preference information. Formally, preference $\mathbf{U}_u$ and content Φ_u^U inputs are first passed through the corresponding DNNs f_U and f_{Φ^U} . Top layer activations are then concatenated together and passed to the fine-tuning network f_U which outputs the latent representation $\hat{\mathbf{U}}_u$. Items are handled in a similar fashion with f_V , f_{Φ^V} and f_U to produce $\hat{\mathbf{V}}_v$. We use separate components for preference and content inputs to handle complex structured content such as images that can't be directly concatenated with preference input in raw form. Another advantage of using a split architecture is that it

allows to use any of the publicly available (or proprietary) pre-trained models for f_{Φ^U} and/or f_{Φ^V} . Training can then be significantly accelerated by updating only the last few layers of each pre-trained network. For domains such as vision where models can exceed 100 layers [11], this can effectively reduce the training time from days to hours. Note that when content input is “compatible” with preference representations we remove f_U and f_{Φ^U} , and directly apply f_U to concatenated input $[\mathbf{U}_u, \Phi_u^U]$. To avoid notation clutter we omit the sub-networks and use f_U and f_V to denote user and item models in subsequent sections.

During training all components are optimized jointly with back-propagation. Once the model is trained we fix it, and make forward passes to map $\mathbf{U} \rightarrow \hat{\mathbf{U}}$ and $\mathbf{V} \rightarrow \hat{\mathbf{V}}$. All retrieval is then done using $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ with relevance scores estimated as before by $\hat{s}_{uv} = \hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T$. Figure 1 shows the full model architecture with both user and item components.

B.4.2 Training For Cold Start

During training we aim to generalize the model to cold start while preserving warm start accuracy. We discussed that existing hybrid model approach this problem by adding additional objective terms and training the model to fall-back on content representations when preferences are not available. However, this complicates learning by forcing the implementer to balance multiple objective terms in addition to training content representations. Moreover, content part of the objective is typically generative forcing the model to explain the observed data instead of using it to maximize recommendation accuracy. This can waste capacity by modeling content aspects that are not useful for recommendations.

We take a different approach and borrow ideas from denoising autoencoders [20] by training the model to reconstruct the input from its corrupted version. The goal is to learn a model that would still produce accurate representations when parts of the input are missing. To achieve this we propose an objective to reproduce the relevance scores after the input is passed through the model:

$$\mathcal{O} = \sum_{u,v} (\mathbf{U}_u \mathbf{V}_v^T - f_U(\mathbf{U}_u, \Phi_u^U) f_V(\mathbf{V}_v, \Phi_v^V))^2 = \sum_{u,v} (\mathbf{U}_u \mathbf{V}_v^T - \hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T)^2$$

O minimizes the difference between scores produced by the input latent model and DNN. When all input is available this objective is trivially minimized by setting the content weights to 0 and learning identity function for preference input. This is a desirable property for reasons discussed below.

In cold start either $\mathbf{U}_u$ or $\mathbf{V}_v$ (or both) is missing so our main idea is to train for this by applying input dropout [18]. We use stochastic mini-batch optimization and randomly sample user-item pairs to compute gradients and update the model. In each mini-batch a fraction of users and items is selected at random and their preference inputs are set to 0 before passing the mini-batch to the model. For “dropped out” pairs the model thus has to reconstruct the relevance scores without seeing the preference input:

$$\begin{aligned} \text{user cold start : } O_{uv} &= (\mathbf{U}_u \mathbf{V}_v^T - f_U(0, \Phi_u^U) f_V(\mathbf{V}_v, \Phi_v^V)^T)^2 \\ \text{item cold start: } O_{uv} &= (\mathbf{U}_u \mathbf{V}_v^T - f_U(\mathbf{U}_u, \Phi_u^U) f_V(0, \Phi_v^V)^T)^2 \end{aligned}$$

Training with dropout has a two-fold effect: pairs with dropout encourage the model to only use content information, while pairs without dropout encourage it to ignore content and simply reproduce preference input. The net effect is balanced between these two extremes. The model learns to reproduce the accuracy of the input latent model when preference data is available while also generalizing to cold start. Dropout thus has a similar effect to hybrid preference-content interpolation objectives but with a much simpler architecture that is easy to optimize. An additional advantage of using dropout is that it was originally developed as a way of regularizing the model. We observe a similar effect here, finding that additional regularization is rarely required even for deeper and more complex models.

There are interesting parallels between our model and areas such as denoising autoencoders [20] and dimensionality reduction [17]. Analogous to denoising autoencoders, our model is trained to reproduce the input from a noisy version. The noise comes in the form of dropout that fully removes a subset of input dimensions. However, instead of reconstructing the actual uncorrupted input we minimize pairwise distances between points in the original and reconstructed spaces. Considering relevance scores $S = \{\mathbf{U}_u \mathbf{V}_v^T \mid u \in U, v \in V\}$ and $\hat{S} = \{\hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T \mid u \in U, v \in V\}$ as sets of points in one dimensional space, the goal is to preserve the relative ordering between the points in $\hat{S}$ produced by our model and the original set S . We focus on

reconstructing distances because it gives greater flexibility allowing the model to learn an entirely new latent space, and not tying it to a representation learned by another model. This objective is analogous to many popular dimensionality reduction models that project the data to a low dimensional space where relative distances between points are preserved [17]. In fact, many of the objective functions developed for dimensionality reduction can also be used here.

A drawback of the objective in Equation 2 is that it depends on the input latent model and thus its accuracy. However, empirically we found this objective to work well producing robust models. The main advantages are that, first, it is simple to implement and has no additional free parameters to tune making it easy to apply to large datasets. Second, in mini-batch mode, NM unique user-item pairs can be sampled to update the networks. Even for moderate size datasets the number of pairs is in the billions making it significantly easier to train large DNNs without over-fitting. The performance is particularly robust on sparse implicit datasets commonly found in CF where $\mathbf{R}$ is binary and over 99% sparse. In this setting training with mini-batches sampled from raw $\mathbf{R}$ requires careful tuning to avoid oversampling 0's, and to avoid getting stuck in bad local optima.

Algorithm 3.1: Learning Algorithm

Input: $\mathbf{R}, \mathbf{U}, \mathbf{V}, \Phi^U, \Phi^V$
Initialize: user model f_U , item model f_V
repeat {DNN optimization }
 sample mini-batch $B = \{(u_1, v_1), \dots, (u_k, v_k)\}$
 for each $(u, v) \in B$ **do**
 apply one of:
 1. **leave as is:**
 2. **user dropout:**
 $[\mathbf{U}_u, \Phi_u^U] \rightarrow [\mathbf{0}, \Phi_u^U]$
 3. **item dropout:**
 $[\mathbf{V}_v, \Phi_v^V] \rightarrow [\mathbf{0}, \Phi_v^V]$
 4. **user transform:**
 $[\mathbf{U}_u, \Phi_u^U] \rightarrow [\text{mean}_{v \in V(u)} \mathbf{V}_v, \Phi_u^U]$
 5. **item transform:**
 $[\mathbf{V}_v, \Phi_v^V] \rightarrow [\text{mean}_{u \in U(v)} \mathbf{U}_u, \Phi_v^V]$
 end for
 update f_U, f_V using B
until convergence
Output: f_U, f_V

附录 C 英文翻译

DropoutNet: 解决推荐系统中的冷启动问题

中文 摘要

由于其性能和可扩展性，潜在模型已成为推荐系统的默认选择。然而，该领域的研究主要集中在对用户-项目交互进行建模，并且很少为冷启动开发潜在模型。

深度学习最近取得了显著的成功，显示出对不同输入类型的出色结果。受这些结果的启发，我们提出了一个名为 DropoutNet 的基于神经网络的潜在模型来解决推荐系统中的冷启动问题。与包含额外的基于内容的客观术语的现有方法不同，我们反而专注于优化并表明神经网络模型可以通过 dropout 明确训练冷启动。我们的模型可以应用于任何现有的潜在模型之上，有效地提供冷启动功能和深度架构的全部功能。根据经验，我们在公开可用的基准上展示了最先进的准确性。代码可在 <https://github.com/layer6ai-labs/DropoutNet> 获得。

C.1 介绍

在线内容交付服务、电子商务和社交网络的流行突出了向消费者展示相关内容的重要挑战。推荐系统已被证明是完成这项任务的有效工具，受到越来越多的关注。构建准确推荐模型的一种常见方法是协同过滤 (CF)。CF 是一种基于其他用户的偏好信息来预测个人偏好的方法。CF 已被证明在各个领域都能很好地工作 [19]，许多成功的网络服务，如 Netflix、Amazon 和 YouTube，都使用 CF 向其用户提供高度个性化的推荐。

CF 中现有的大多数方法可以分为两类：基于邻居的和基于模型的。基于模型的方法，尤其是潜在模型，通常是首选，因为它们构建了数据的紧凑表示并实现了高精度。这些表示针对快速检索进行了优化，并且可以扩展以实时处理数百万用户。由于这些原因，我们专注于这项工作中的潜在方法。潜在模型通常是通过目标偏好矩阵应用低秩近似的变体来学习的。因此，当有大量偏好信息可用时，它们运行良好，但在高度稀疏的设置中开始降级。当没有可用于给定用户或项目的偏好信息时，会发生称为冷启动的最极端的稀疏情况。在这种情况下，生成个

性化推荐的唯一方法是合并其他内容信息。基础潜在方法不能包含内容，因此已经提出了许多混合模型 [3, 21, 22] 来组合偏好和内容信息。然而，大多数混合方法引入了额外的客观术语，使学习和推理变得相当复杂。此外，目标的内容部分通常是生成性的 [21、9、22]，迫使模型“解释”内容而不是使用它来最大化推荐准确性。

最近，深度学习在计算机视觉 [15, 11]、语音 [12, 10] 和自然语言处理 [5, 16] 等领域取得了显著的成功。在所有这些领域中，端到端深度神经网络 (DNN) 模型在几乎没有特征工程的情况下实现了最先进的准确性。这些结果表明，深度学习在推荐系统的内容建模方面也应该非常有效。然而，虽然最近在将深度学习应用于 CF [7、22、6、23] 方面取得了一些进展，但关于使用深度学习解决冷启动问题的研究却很少。

在这项工作中，我们提出了一个模型来解决这一差距。我们的方法基于冷启动等同于偏好信息缺失的缺失数据问题的观察。因此，我们没有向模型内容添加额外的客观术语，而是修改学习过程以明确地为缺失的输入调节模型。我们方法背后的关键思想是，通过将 dropout [18] 应用于输入小批量，我们可以训练 DNN 泛化到缺失的输入。通过选择适当数量的 dropout，我们表明可以学习基于 DNN 的潜在模型，该模型在热启动时的性能与最先进的模型相当，而在冷启动时的性能明显优于它。生成的模型比大多数混合方法更简单，并使用单个目标函数，联合优化所有组件以最大限度地提高推荐准确性。

我们方法的另一个优点是它可以应用于任何现有的潜在模型之上，以提供/增强其冷启动能力。这几乎不需要修改原始模型，从而最大限度地减少了任何已经运行潜在模型的生产环境的实施障碍。在以下部分中，我们详细描述了我们的方法，并展示了公开可用基准的实证结果。

C.2 框架

在典型的 CF 问题中，我们有一组为 N 的用户 $U = \{u_1, \dots, u_N\}$ 和一组为 M 的项目 $V = \{v_1, \dots, v_M\}$ 。用户对物品的反馈可以用 $N \times M$ 的偏好矩阵 $\mathbf{R}$ 表示，其中 $\mathbf{R}_{uv}$ 是用户 u 对物品 v 的偏好。 $\mathbf{R}_{uv}$ 可以由用户以评分、喜欢/不喜欢等形式明确提供，也

可以从观看、播放和购买等隐式交互中推断出来。在显式设置 $\mathbf{R}$ 中通常包含分级相关性（例如，1-5 评级），而在隐式设置 $\mathbf{R}$ 中通常是二元的；我们在这项工作中考虑了这两种情况。当没有偏好信息可用时 $\mathbf{R}_{uv} = 0$ 。我们用 $U(v) = \{u \in U \mid \mathbf{R}_{uv} \neq 0\}$ 来表示偏好的用户集，并用 $V(u) = \{v \in V \mid \mathbf{R}_{uv} \neq 0\}$ 表示偏好的项目集。在冷启动中，没有可用的偏好信息，我们定义 $V(u) = \emptyset$ 和 $U(v) = \emptyset$ 给用户 u 和项目 v 。

此外，在许多领域中，我们通常可以访问用户和项目的内容信息。对于项目，此信息可以文本、音频或图像/视频的形式出现。对于用户，我们可以拥有个人资料信息（年龄、性别、位置、设备等）和社交媒体数据（Facebook、Twitter 等）。此数据可以为推荐模型提供非常有用的信号，并且在偏好信息很少或没有可用的稀疏和冷启动设置中特别有效。在应用相关变换后，大多数内容信息可以用固定长度的特征向量表示。我们使用 Φ^U 和 Φ^V 分别表示用户和项目的内容特征，其中 $\Phi_u^U(\Phi_v^V)$ 是用户（项目）的内容特征向量。当内容缺失时，相应的特征向量设置为 0。目标是将偏好信息 $\mathbf{R}$ 与内容特征 Φ^U 和 Φ^V 一起使用，以学习准确且稳健的推荐模型。理想情况下，该模型应处理用户/项目的所有阶段：从冷启动到早期稀疏偏好，再到后期明确定义的偏好配置文件。

C.3 相关工作

许多混合潜在方法已经被提出用来解决 CF 中的冷启动问题。一种比较流行的模型是协作主题回归 (CTR) [21]，它结合了潜在狄利克雷分配 (LDA) [4] 和加权矩阵分解 (WMF) [13]。当首选项可用时，CTR 在冷启动和 WMF 中的 LDA 表示之间进行插值。最近，已经提出了几种相关的方法。协作主题泊松分解 (CTPF) [8] 使用类似的插值架构，但用泊松分解 [9] 替换了 LDA 和 WMF 组件。协作深度学习 (CDL) [22] 是另一种具有类似架构的方法，其中 LDA 被堆叠式去噪自动编码器 [20] 取代。

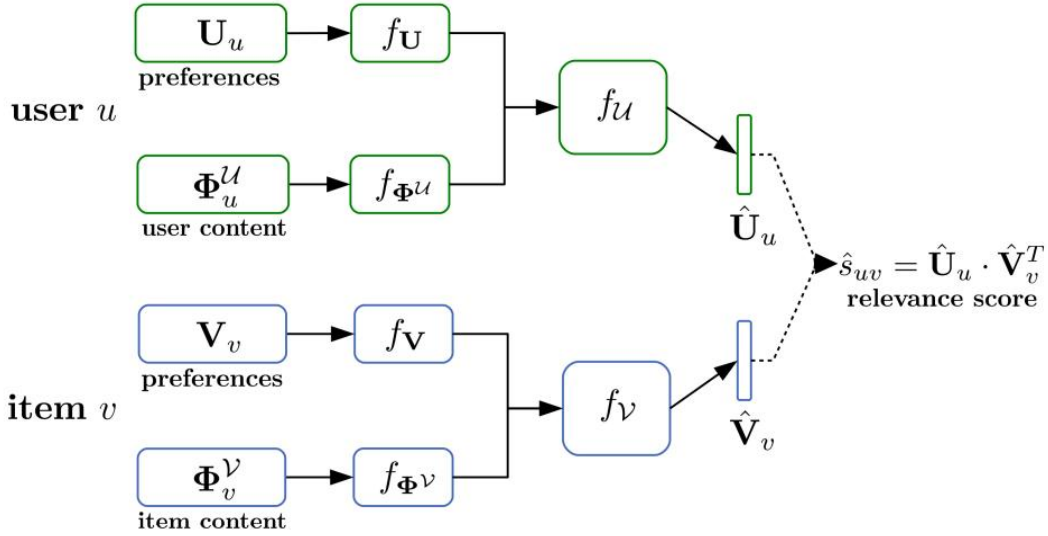


图 1: DropoutNet 架构图。对于每个用户 u ，偏好 $\mathbf{U}_u$ 和内容 Φ_u^U 输入首先通过相应的 DNN: f_U 和 f_{Φ^U} 。然后将顶层激活连接在一起并传递给输出潜在表示 $\hat{\mathbf{U}}_u$ 的微调网络 f_u 。项目以类似的方式处理 f_V, f_{Φ^V}, f_v 和生产 $\hat{\mathbf{V}}_v$ 。所有组件都与反向传播联合优化，然后在推理过程中保持固定。步骤是在新的潜在空间中完成的，使用 $\hat{\mathbf{U}}$ 和 $\hat{\mathbf{V}}$ 替换原始表示 $\mathbf{U}$ 和 $\mathbf{V}$ 。

虽然这些模型实现了极具竞争力的性能，但它们也有一些缺点。首先，它们将偏好和内容成分都纳入了目标函数，使其变得非常复杂。例如，CDL 包含四个客观术语，除了 WMF 和自动编码器参数外，还需要调整三个组合权重。这使得在每个参数设置实验都昂贵且耗时的大型数据集上调整这些模型变得具有挑战性。其次，各模型的制定均假设了冷启动项，不适用于冷启动用户。大多数在线服务必须经常合并新用户和新项目，因此需要能够处理这两者的模型。原则上，可以为用户推导出一个类似的模型并联合优化这两个模型。然而，这将需要一个更复杂的目标，几乎使自由参数的数量增加一倍。我们旨在通过这项工作解决的主要问题之一是我们是否开发了一个适用于用户和项目的更简单的冷启动模型？

除了 CDL 之外，还提出了许多方法来利用 DNN 进行 CF。DeepMusic [7] 的早期方法之一旨在预测由仅使用内容 DNN 的潜在模型学习的潜在表示。最近，[6] 描述了 YouTube 的两阶段推荐模型，该模型将用户会话（最近的播放和搜索）和个人资料信息作为输入。给定会话中项目的潜在表示被平均，与配置文件信息连接，并传递给 DNN，后者为用户输出依赖于会话的潜在表示。平均项目解决了

可变长度输入问题，但可能会丢失会话的时间方面。为了更准确地模拟用户的偏好如何随时间变化，[23] 提出了一种递归神经网络 (RNN) 方法。RNN 一次按顺序应用于一个项目，在处理完所有项目后，隐藏层激活用作潜在表示。

其中许多模型显示了将深度架构应用于 CF 的明显好处。然而，当内容信息可用时，很少有人研究冷启动和稀疏设置性能。可以说，我们期望深度学习在这些场景中最有益，因为它对各种内容类型具有出色的泛化能力。我们提出的方法旨在利用这一优势，并且与 [6] 最为相似。我们还使用潜在表示作为用户和项目的偏好特征输入，并将它们与内容结合以训练基于 DNN 的混合模型。但与主要关注热启动用户的 [6] 不同，我们为用户和项目开发了类似的模型，然后展示了如何训练这些模型以明确处理冷启动。

C.4 我们的方法

在本节中，我们将描述我们称为 DropoutNet 的模型的架构，以及学习和推理过程。我们从输入表示开始。我们的目标是开发一个能够处理冷启动和热启动场景的模型。因此，模型的输入需要包含内容和偏好信息。一种选择是直接使用原始形式 $\mathbf{R}$ 的行和列。然而，随着用户和项目数量的增长，这些变得非常大。相反，我们采用与 [6] 和 [23] 类似的方法，并使用潜在表示作为偏好输入。潜在模型通常使用低秩矩阵 $\mathbf{U}$ 和 $\mathbf{V}$ 的乘积来近似偏好矩阵：

$$\mathbf{R}_{uv} \approx \mathbf{U}_u \mathbf{V}_v^T$$

其中 $\mathbf{U}_u$ 和 $\mathbf{V}_v$ 分别是用户 u 和项目 v 的潜在表示。 $\mathbf{U}$ 和 $\mathbf{V}$ 都是稠密和低维的 $D \ll \min(M, N)$ 。注意到潜在方法在广泛的 CF 数据集上的强大性能，假设潜在表示准确地总结了有关用户和项目的偏好信息就足够了。此外，低输入维数显著降低了 DNN 的模型复杂性，因为第一个隐藏层的激活大小与输入大小成正比。鉴于这些优势，我们分别为每个用户和项目设置输入 $[\mathbf{U}_u, \Phi_u^U]$ 和 $[\mathbf{V}_v, \Phi_v^V]$ 。

C.4.1 模型架构

鉴于联合偏好内容输入，我们建议应用 DNN 模型将其映射到包含内容和偏好信息的新潜在空间。形式上，偏好 $\mathbf{U}_u$ 和内容 Φ_u^U 输入首先通过相应的 DNN: f_U

和 f_{Φ_U} ，然后将顶层激活连接在一起并传递给输出潜在表示 $\hat{\mathbf{U}}_u$ 的微调网络 f_U 。项目以类似的方式处理 f_v, f_{Φ_v}, f_V 和生产 $\hat{\mathbf{V}}_v$ 。我们对偏好和内容输入使用单独的组件来处理复杂的结构化内容，例如无法直接与原始形式的偏好输入连接的图像。使用拆分架构的另一个优点是它允许使用任何公开可用的（或专有的）预训练模型 f_{Φ_U} 和/或 f_{Φ_v} 。然后可以通过仅更新每个预训练网络的最后几层来显著加速训练。对于模型可以超过 100 层的视觉领域 [11]，这可以有效地将训练时间从几天缩短到几小时。请注意，当内容输入与偏好表示“兼容”时，我们删除 f_U 和 f_{Φ_U} ，并直接应用于 f_U 来连接的输入 $[\mathbf{U}_u, \Phi_u^U]$ 。为了避免符号混乱，我们在后续部分中省略了子网络并使用 f_U 和 f_v 来表示用户和项目模型。

在训练期间，所有组件都通过反向传播进行优化。一旦模型被训练好，我们就固定它，并通过 $\mathbf{U} \rightarrow \hat{\mathbf{U}}$ 和 $\mathbf{V} \rightarrow \hat{\mathbf{V}}$ 向前传递。然后使用 $\hat{\mathbf{U}}$ 和 $\hat{\mathbf{V}}$ 与之前通过 $\hat{s}_{uv} = \hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T$ 估计的相关性分数一起完成所有步骤。图 1 显示了包含用户和项目组件的完整模型架构。

C.4.2 冷启动训练

在训练期间，我们的目标是将模型推广到冷启动，同时保持热启动精度。我们讨论了现有的混合模型通过添加额外的客观术语并训练模型在偏好不可用时回退到内容表示来解决这个问题。然而，这会迫使实施者在训练内容表示之外平衡多个客观术语，从而使学习变得复杂。此外，目标的内容部分通常是生成性的，迫使模型解释观察到的数据，而不是使用它来最大化推荐准确性。这会通过对对推荐无用的内容方面进行建模来浪费容量。

我们采用不同的方法并通过训练模型从其损坏版本重建输入来借鉴去噪自动编码器 [20] 的想法。目标是学习一种模型，该模型在部分输入缺失时仍能产生准确的表示。为实现这一目标，我们提出了一个目标，即在输入通过模型后重现相关性分数：

$$\mathcal{O} = \sum_{u,v} (\mathbf{U}_u \mathbf{V}_v^T - f_U(\mathbf{U}_u, \Phi_u^U) f_v(\mathbf{V}_v, \Phi_v^V))^2 = \sum_{u,v} (\mathbf{U}_u \mathbf{V}_v^T - \hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T)^2$$

O 最小化输入潜在模型和 DNN 产生的分数之间的差异。当所有输入都可用时，通过将内容权重设置为 0 并学习偏好输入的身份函数，可以将这个目标最小化。由于下面讨论的原因，这是一个理想的特性。

在冷启动中，其中一个 $\mathbf{U}_u$ 或 $\mathbf{V}_v$ （或两个）缺失，所以我们的主要想法是通过应用输入丢失来训练这个 [18]。我们使用随机小批量优化和随机采样用户-项目对来计算梯度和更新模型。在每个 mini-batch 中，随机选择一小部分用户和项目，并且在将 mini-batch 传递给模型之前将他们的偏好输入设置为 0。因此，对于“退出”对，模型必须在不查看偏好输入的情况下重建相关性分数：

$$\begin{aligned} \text{user cold start: } O_{uv} &= (\mathbf{U}_u \mathbf{V}_v^T - f_U(0, \Phi_u^U) f_V(\mathbf{V}_v, \Phi_v^V))^2 \\ \text{item cold start: } O_{uv} &= (\mathbf{U}_u \mathbf{V}_v^T - f_U(\mathbf{U}_u, \Phi_u^U) f_V(0, \Phi_v^V))^2 \end{aligned}$$

使用 dropout 进行训练有双重效果：使用 dropout 的配对鼓励模型仅使用内容信息，而没有使用 dropout 的配对则鼓励它忽略内容并简单地重现偏好输入。净效应在这两个极端之间取得平衡。该模型学习在偏好数据可用时重现输入潜在模型的准确性，同时也泛化到冷启动。因此，Dropout 具有与混合偏好内容插值目标类似的效果，但具有更简单且易于优化的架构。使用 dropout 的另一个优点是它最初是作为一种正则化模型的方式开发的。我们在这里观察到类似的效果，发现即使对于更深和更复杂的模型，也很少需要额外的正则化。

我们的模型与去噪自动编码器 [20] 和降维 [17] 等领域之间存在有趣的相似之处。类似于去噪自动编码器，我们的模型经过训练可以从嘈杂的版本中再现输入。噪声以 dropout 的形式出现，它完全删除了输入维度的一个子集。然而，我们不是重建实际未损坏的输入，而是最小化原始空间和重建空间中点之间的成对距离。考虑到相关性分数一维空间中的点集 $S = \{\mathbf{U}_u \mathbf{V}_v^T \mid u \in U, v \in V\}$ 和 $\hat{S} = \{\hat{\mathbf{U}}_u \hat{\mathbf{V}}_v^T \mid u \in U, v \in V\}$ ，目标是保持我们的模型产生的点 $\hat{S}$ 与原始点集 S 之间的相对顺序。我们专注于重建距离，因为它提供了更大的灵活性，允许模型学习一个全新的潜在空间，而不是将其与另一个模型学习的表示联系起来。这个目标类似于许多流行的降维模型，这些模型将数据投影到保留点之间相对距离的低维空间 [17]。事实上，这里也可以使用许多为降维而开发的目标函数。

上述公式中目标的一个缺点是它取决于输入潜在模型，因此取决于它的准确性。然而，根据经验，我们发现这个目标可以很好地生成稳健的模型。主要优点

是，首先，它实现简单，没有额外的自由参数可以调整，因此很容易应用于大型数据集。其次，在小批量模式下，可以对 NM 个唯一的用户-项目对进行采样以更新网络。即使对于中等大小的数据集，对的数量也有数十亿，这使得在不过度拟合的情况下训练大型 DNN 变得非常容易。该性能在 CF 中常见的稀疏隐式数据集上特别稳健，其中二进制数据集 $\mathbf{R}$ 超过 99% 稀疏。在此设置中，使用从原始样本 $\mathbf{R}$ 中采样的小批量进行训练需要仔细调整以避免过度采样 0，并避免陷入糟糕的局部最优。

 Algorithm 3.1: Learning Algorithm

Input: $\mathbf{R}, \mathbf{U}, \mathbf{V}, \Phi^U, \Phi^V$
Initialize: user model f_U , item model f_V
repeat {DNN optimization }
 sample mini-batch $B = \{(u_1, v_1), \dots, (u_k, v_k)\}$
 for each $(u, v) \in B$ **do**
 apply one of:
 1. **leave as is:**
 2. **user dropout:**
 $[\mathbf{U}_u, \Phi_u^U] \rightarrow [\mathbf{0}, \Phi_u^U]$
 3. **item dropout:**
 $[\mathbf{V}_v, \Phi_v^V] \rightarrow [\mathbf{0}, \Phi_v^V]$
 4. **user transform:**
 $[\mathbf{U}_u, \Phi_u^U] \rightarrow [\text{mean}_{v \in V(u)} \mathbf{V}_v, \Phi_u^U]$
 5. **item transform:**
 $[\mathbf{V}_v, \Phi_v^V] \rightarrow [\text{mean}_{u \in U(v)} \mathbf{U}_u, \Phi_v^V]$
 end for
 update f_U, f_V using B
until convergence
Output: f_U, f_V
